Data structures and Algorithms

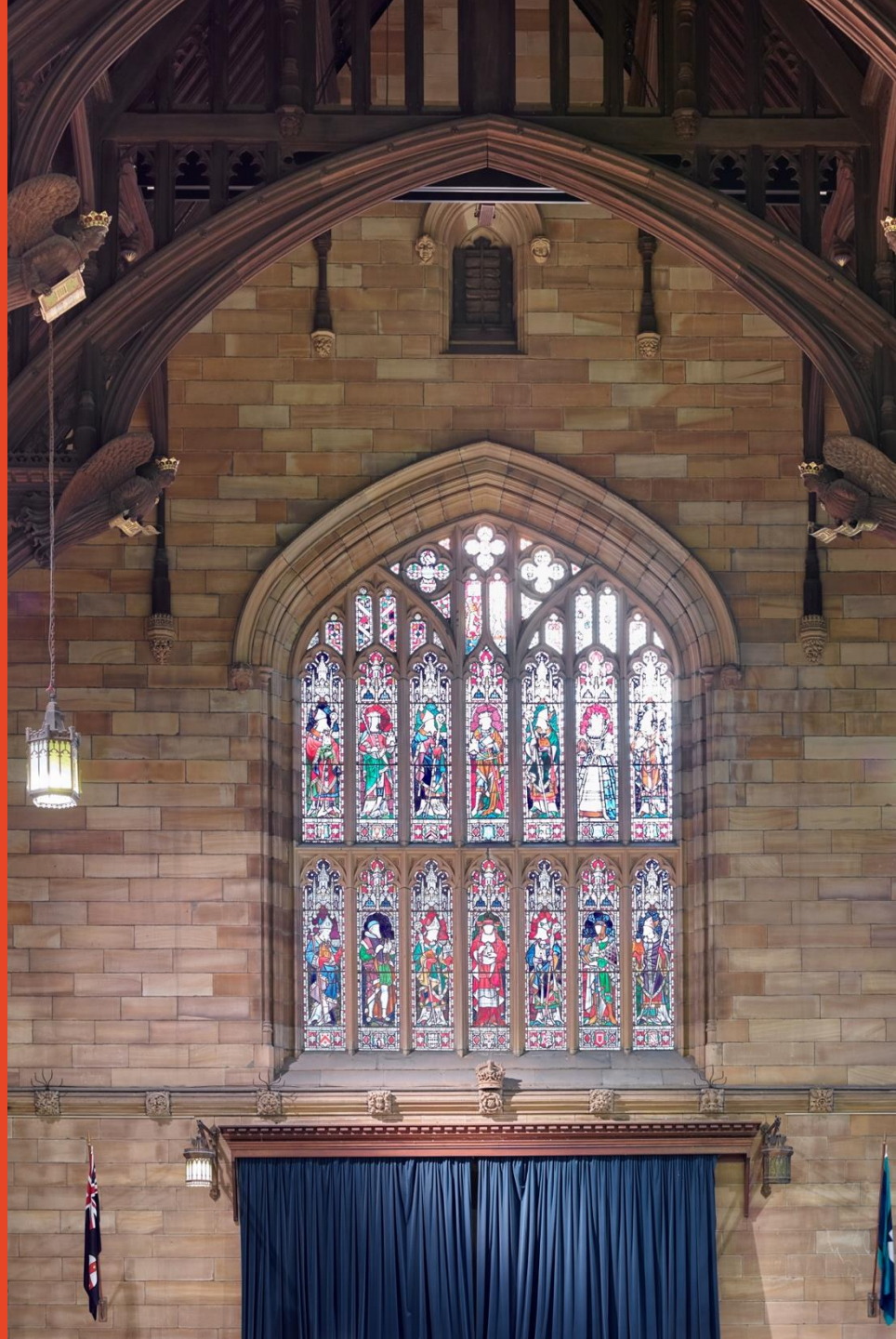
Binary Search Trees

version 1.0

Dr. Armin Chitizadeh
School of Computer Science

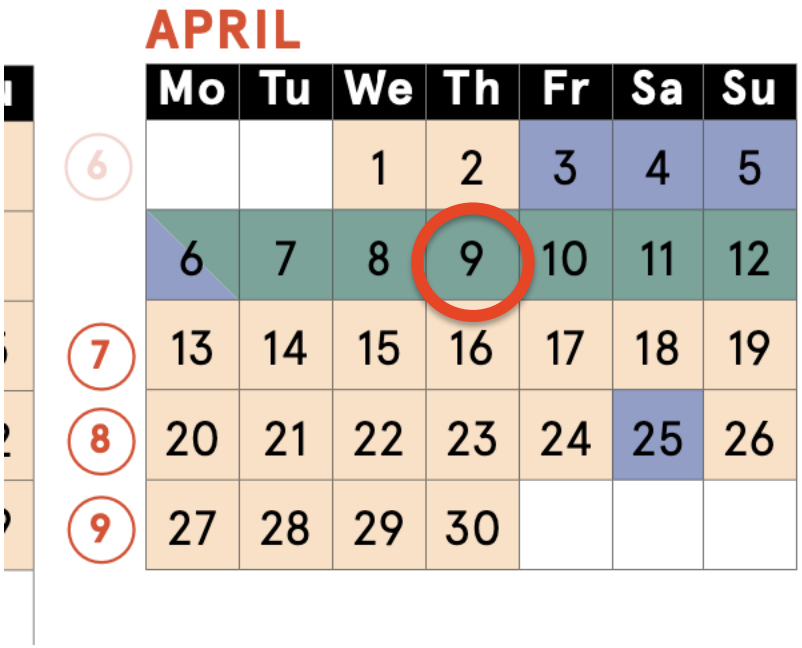
Inspired by lectures by Dr. Karlos Ishac and Dr. Andre van Renssen.

THE UNIVERSITY OF SYDNEY

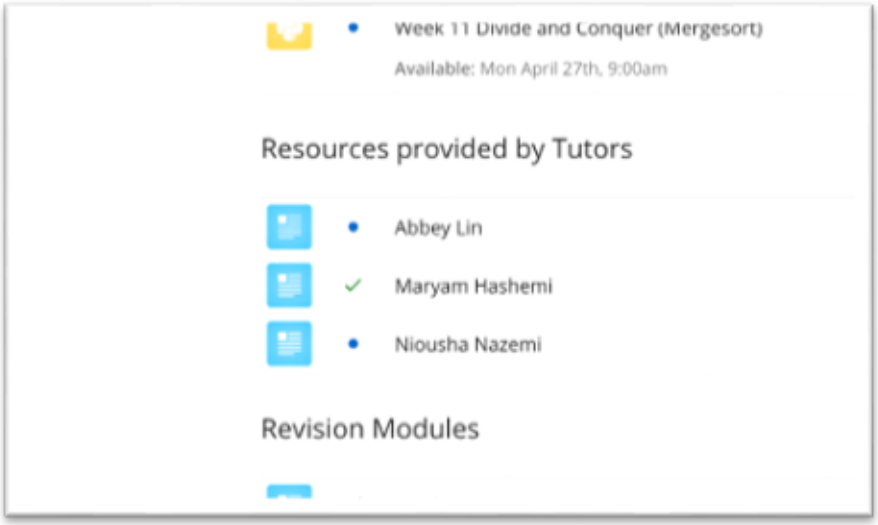


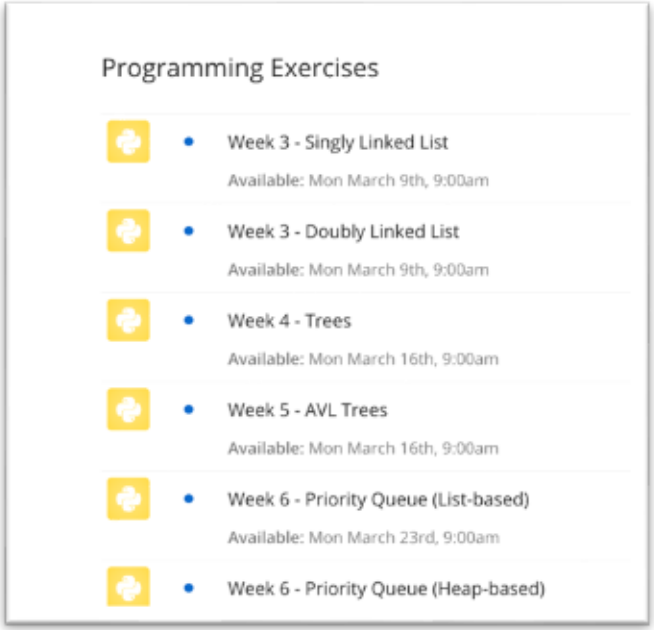
Mid Semester Review Session

– Thursday



There are some great resources, utilise them!





https://edstem.org/au/courses/30783/lessons

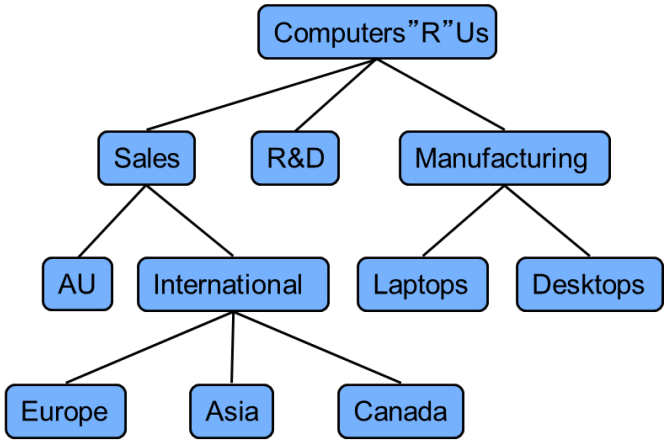
Recap

- Trees
- Big O Examples
- Iterator and positions

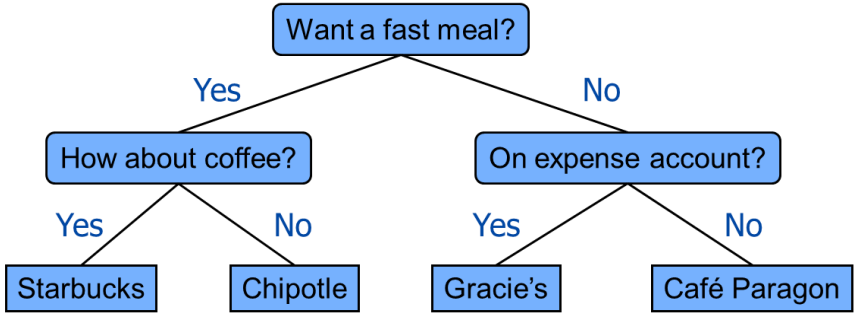
© Goodrich, Tamassia, Goldwasser

Trees

- General Trees vs Binary Trees
- Traversals: Preorder, Postorder, Inorder
- Euler Tour



General Tree



Binary Tree

© Goodrich, Tamassia, Goldwasser

Big O Examples

- Big O (O()) describes the upper bound of the complexity.
- Omega (Ω()) describes the lower bound of the complexity.

- Basic Operations:
  - single arithmetic operation (e.g., +, *).
  - single assignment (e.g. x := 0)
  - single test (e.g., x == 0)
  - single read (of a primitive type: integer, float, character, bool)
  - single write (of a primitive type: integer, float, character, bool)

© Goodrich, Tamassia, Goldwasser

Big O Examples

(1) //Simple Assignment

```
X = 0 // O(1)
```

(2) //Series of Statements

```
X = 0 // O(1)
Y = 2 // O(1)
Z = X + Y // O(1)
```

(3) //If Conditionals

```
if (cond) then
    block 1 // O(1)
else
    block 2 // O(1)
end if;
```

(4) //If Conditionals

```
if (cond) then
    block 1 // O(1)
else
    block 2 // O(n)
end if;
```


© Goodrich, Tamassia, Goldwasser

Big O Examples

(5) //Loop

```
for x in 1…N:
    statements // O(1)
end
```

(6) //Complex Loop

```
for x in 1…N:
    statements // O(n)
end
```

(7) //Nested Loop

```
for x in 1…N:
    for y in 1…M:
        statements // O(1)
    end
end
```

(8) //Complex Nested Loop

```
for x in 1…N:
    for y in 1…M:
        statements // O(n^2)
    end
end
```


© Goodrich, Tamassia, Goldwasser

Big O Examples

(9) //Binary Search def

```
binary_search(dictionary, word):
    low, high = 0, len(dictionary) - 1
    while low <= high:
        mid = (low + high)
        if dictionary[mid] ==word:
            return f"Found: {word}"
        elif dictionary[mid] < word:
            low = mid + 1
        else:
            high = mid - 1
    return "Not Found"
```

(10) //Managing a Warehouse

```
for shelf in range(n):
        for row in range(n):
            for column in range(n):
                total_items +=
                    warehouse[shelf][row][column]
    return total_items
```


© Goodrich, Tamassia, Goldwasser

<BIG-O-CHEATSHEET>

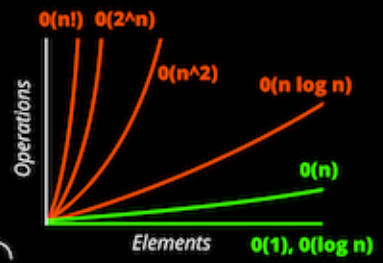
www.bigocheatsheet.com

LEGEND

TIME Complexity VS. SPACE Complexity

Good Fair Bad

Good Fair Bad



DATA STRUCTURE Operations

DATA Structure	TIME Complexity Average: Access	Search	Insertion	Deletion	TIME Complexity Worst: Access	Search	Insertion	Deletion	SPACE Complexity Worst
Array	Θ(1)	Θ(n)	Θ(n)	Θ(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Stack	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Singly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Doubly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Skip List	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n log(n))
Hash Table	N/A	Θ(1)	Θ(1)	Θ(1)	N/A	O(n)	O(n)	O(n)	O(n)
Binary Search Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)
Cartesian Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(n)	O(n)	O(n)	O(n)
B-Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Red-Black Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Splay Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(log(n))	O(log(n))	O(log(n))	O(n)
AVL Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
KD Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)

ARRAY SORTING Algorithms

ARRAY Algorithms	TIME Complexity Best	Average	Worst	SPACE Complexity Worst
Quicksort	Ω(n log(n))	Θ(n log(n))	O(n^2)	O(log(n))
Mergesort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(n)
Timsort	Ω(n)	Θ(n log(n))	O(n log(n))	O(1)
Heapsort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(1)
Bubble Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Insertion Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Selection Sort	Ω(n^2)	Θ(n^2)	O(n^2)	O(1)
Tree Sort	Ω(n log(n))	Θ(n log(n))	O(n^2)t	O(n)
Shell Sort	Ω(n log(n))	Θ(n(log(n))^2)	O(n(log(n))^2)	O(1)
Bucket Sort	Ω(n+k)	Θ(n+k)	O(n^2)	O(n)
Radix Sort	Ω(nk)	Θ(nk)	O(nk)	O(n+k)
Counting Sort	Ω(n+k)	Θ(n+k)	O(n+k)	O(k)
Cubesort	Ω(n)	Θ(n log(n))	O(n log(n))	O(n)

Iterator and Iterable

1. Iterator iterator()

This method returns an iterator that allows traversal of the tree's elements in a predefined order (e.g., in-order, pre-order, or post-order traversal).

```
tree = BinaryTree(root)
print("In-order:", list(tree.iterator("inorder")))   # [2, 5, 7, 10, 15]
print("Pre-order:", list(tree.iterator("preorder"))) # [10, 5, 2, 7, 15]
```

2. Iterable positions()

This method returns an Iterable collection of tree positions (nodes).

```
tree = BinaryTree(root)
print("In-order positions:", [node.value for node in tree.positions("inorder")])
# Output: [2, 5, 7, 10, 15]
```


© Goodrich, Tamassia, Goldwasser

Binary Search Trees

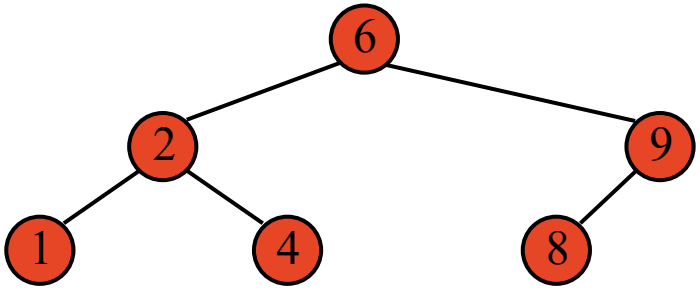
Binary Search Trees (BST)

A binary search tree is a binary tree storing keys (or key-value pairs) satisfying the following BST property

For any node v in the tree and
any node u in the left subtree of v and
any node w in the right subtree of v,

key(u) < key(v) < key(w)

Note that an inorder traversal of a binary search tree visits the keys in increasing order.

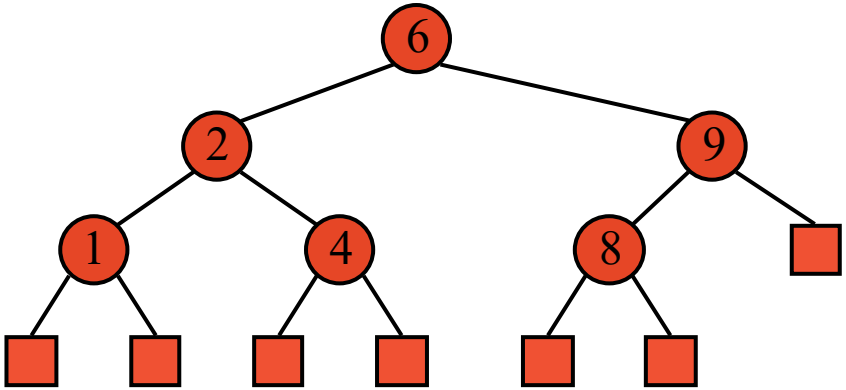


© Goodrich, Tamassia, Goldwasser

BST Implementation

To simplify the presentation of our algorithms, we only store keys (or key-value pairs) at internal nodes

External nodes do not store items (and with careful coding, can be omitted, using null to refer to such)



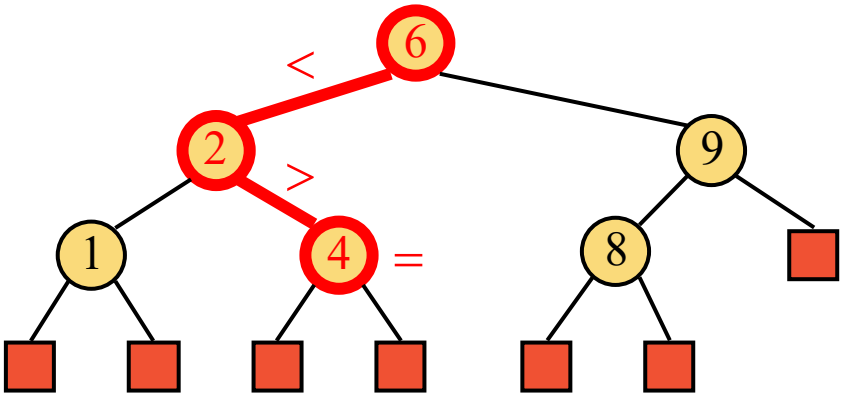
© Goodrich, Tamassia, Goldwasser

Searching with a Binary Search Tree

To search for a key k, we trace a downward path starting at the root

To decide whether to go left or right, we compare the key of the current node v with k

If we reach an external node, this means that the key is not in the data structure

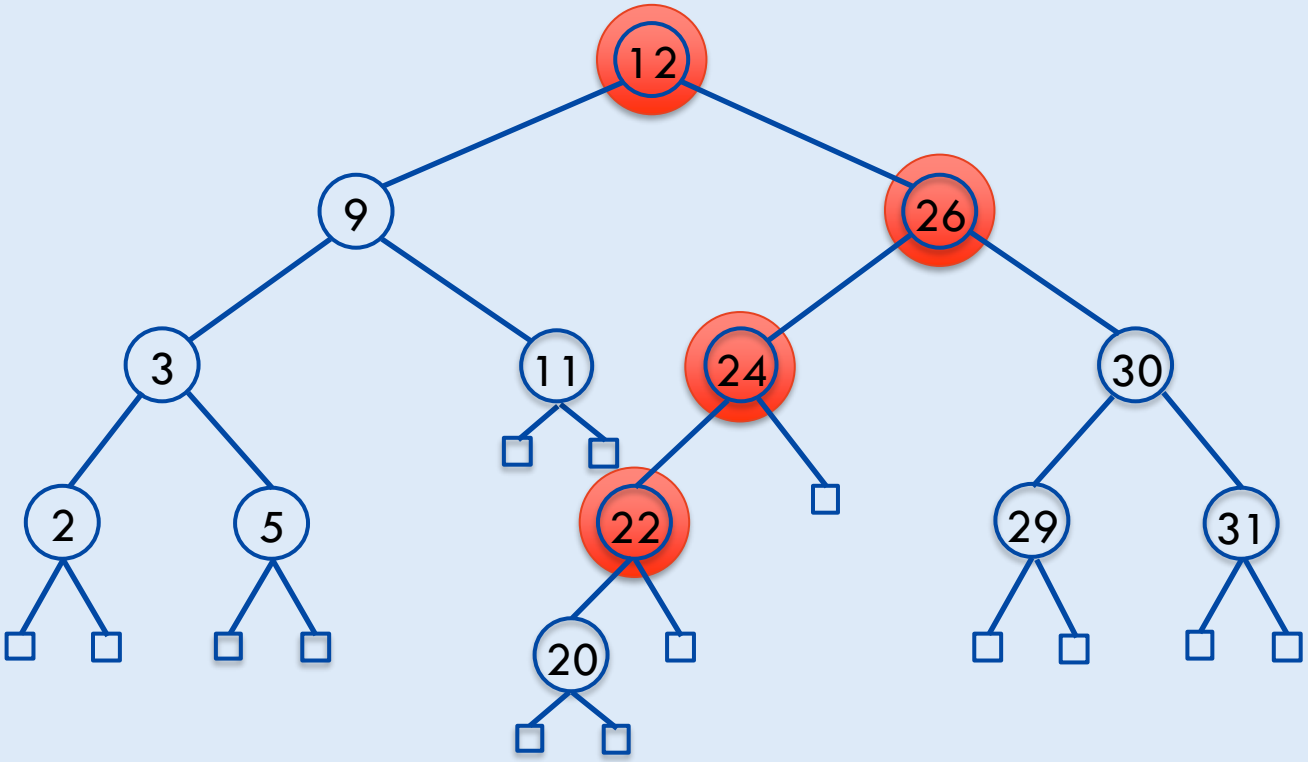



```
def search(k, v)
    if v.isExternal() then
        # unsuccessful search
        return v
    if k = key(v) then
        # successful search
        return v
    else if k < key(v) then
        # recurse on left subtree
        return search(k, v.left)
    else
        # that is  k > key(v)
        # recurse on right subtree
        return search(k, v.right)
```


© Goodrich, Tamassia, Goldwasser

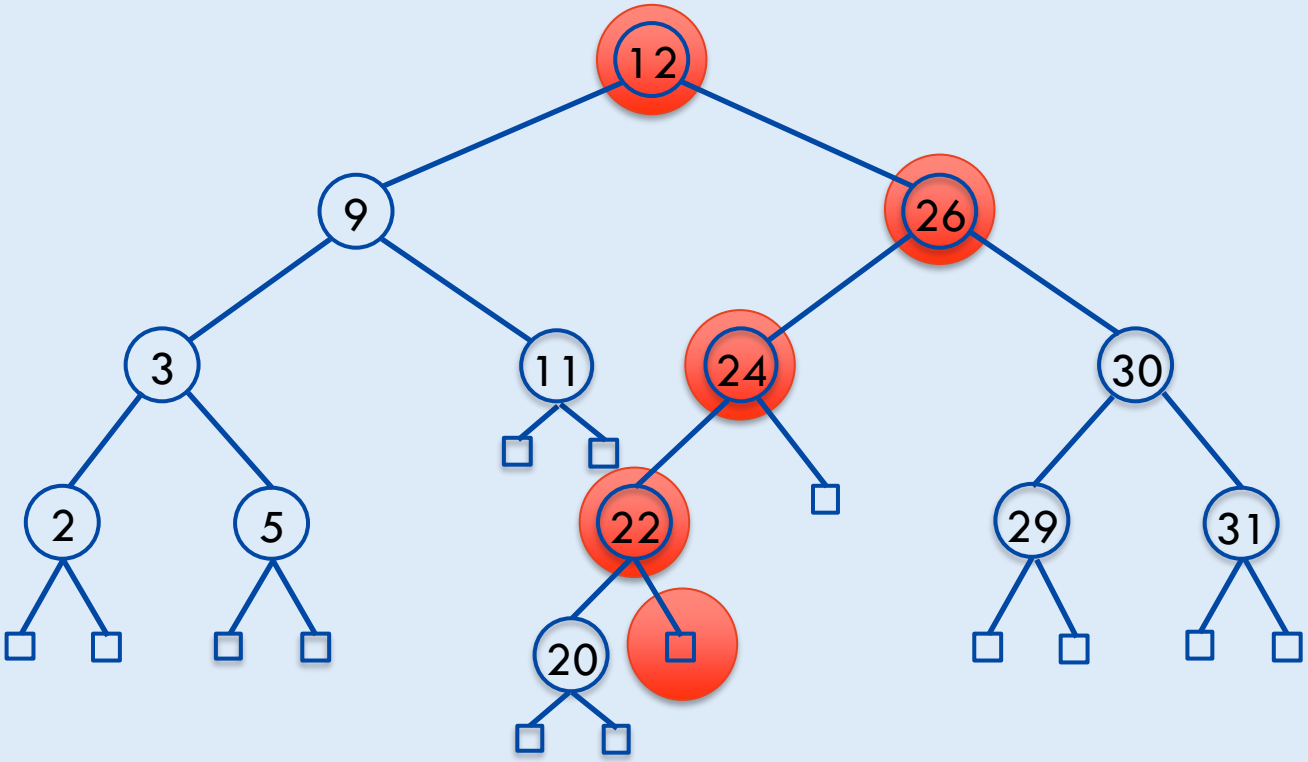
Interactive Example: Find 22

S={2,3,5,9,11,12,20,22,24,26,29,30,31}



Interactive Example: Find 23

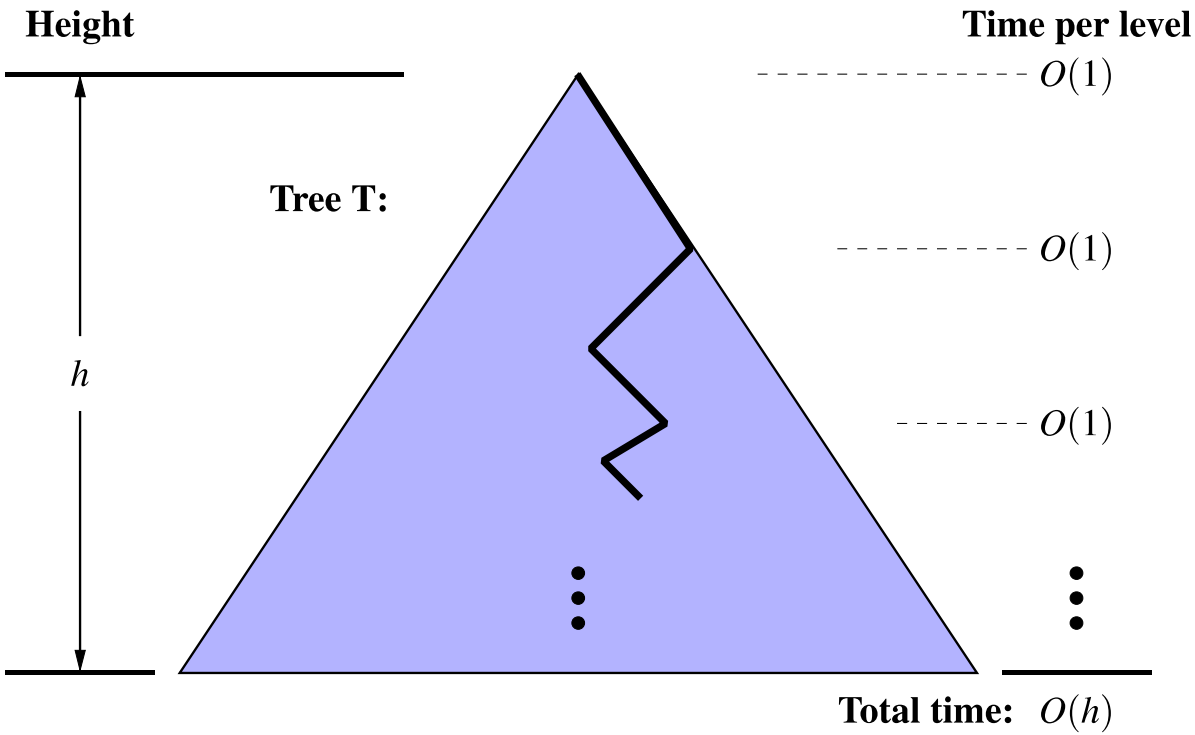
S={2,3,5,9,11,12,20,22,24,26,29,30,31}



Analysis of Binary Tree Searching

Runs in O(h) time, where h is the height of the tree

- worst case is $h = n - 1$
- best case is $h \leq \log_2 n$



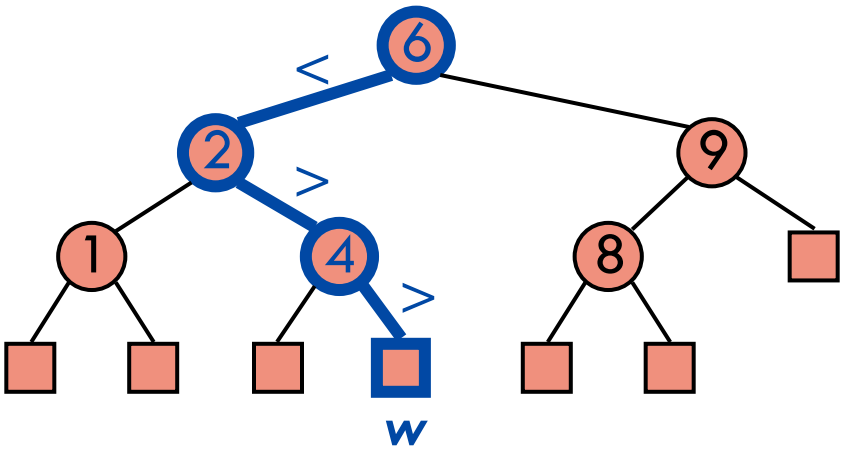
© Goodrich, Tamassia, Goldwasser

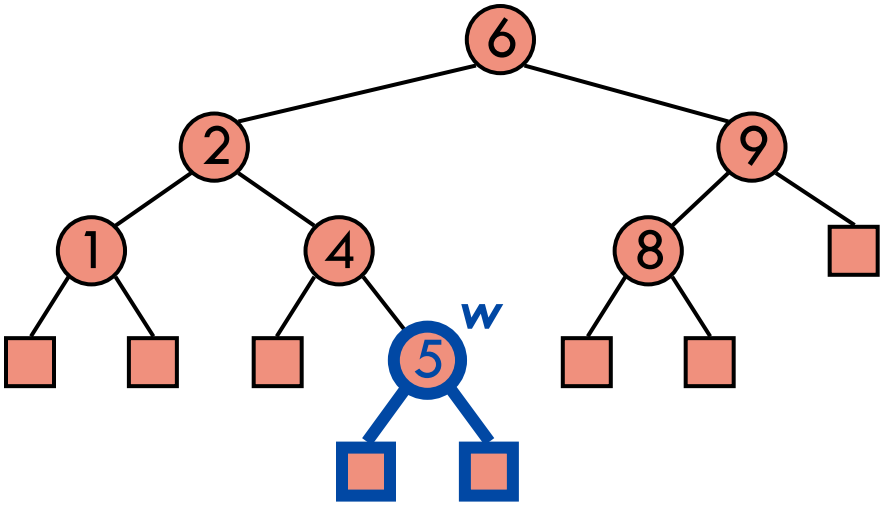
Insertion

To perform operation put(k, o), we search for key k (using search)

If k is found in the tree, replace the corresponding value by o

If k is not found, let w be the external node reached by the search. We replace w with an internal node holding (k, o)

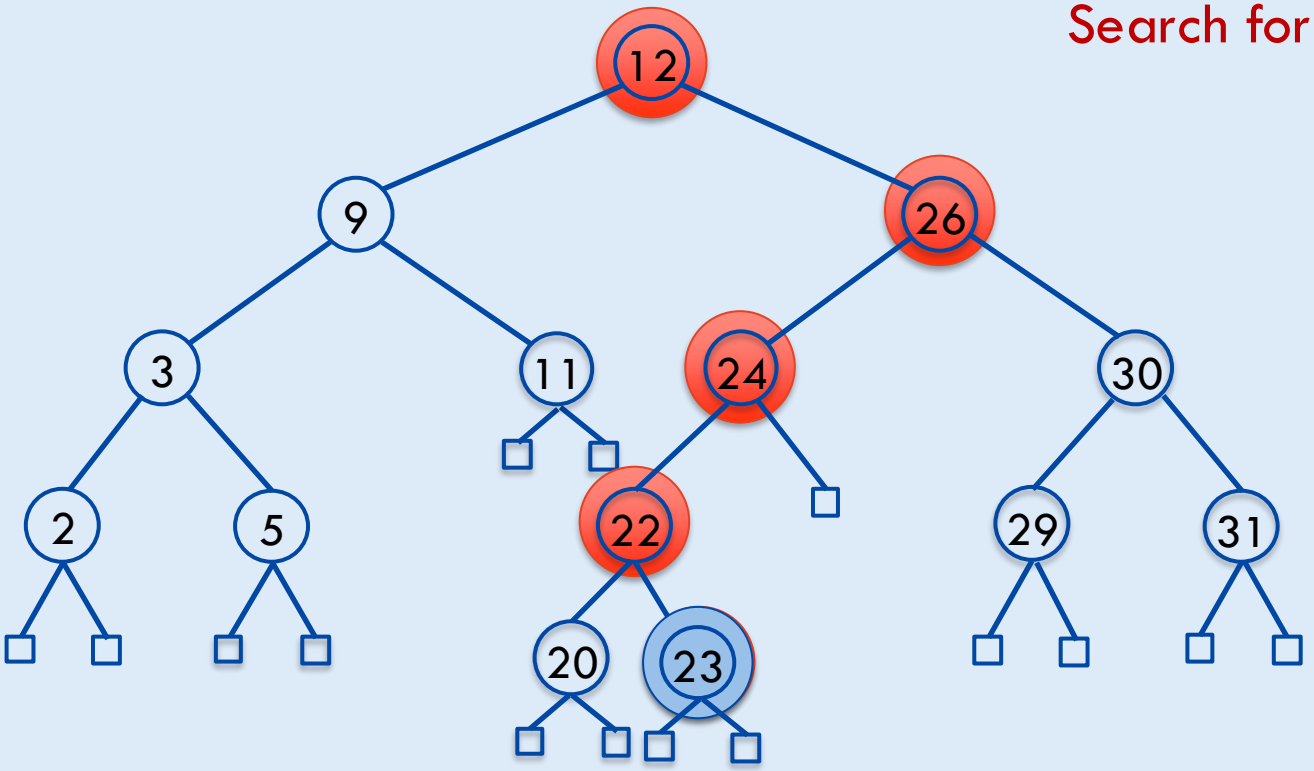




Interactive Example: Insert 23

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Search for 23



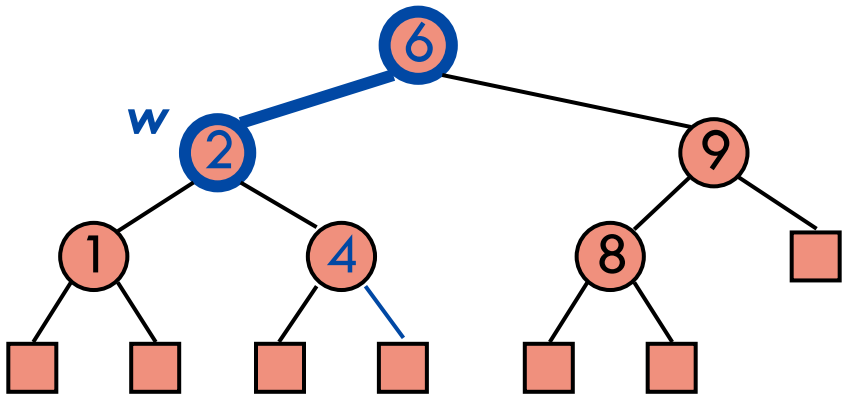
Delete

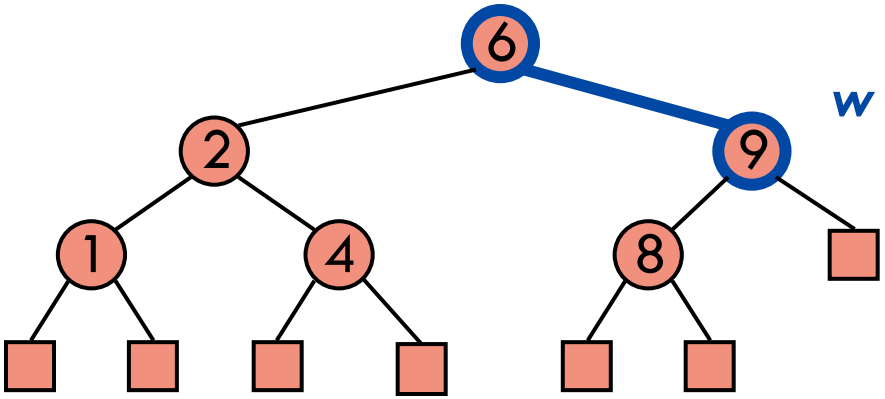
To perform operation remove(k), we search for key k (using search) to find the node w holding k

We distinguish between two cases

- w has one external child
- w has two internal children

If k is not in the tree we can either throw an exception or do nothing depending on the ADT specs





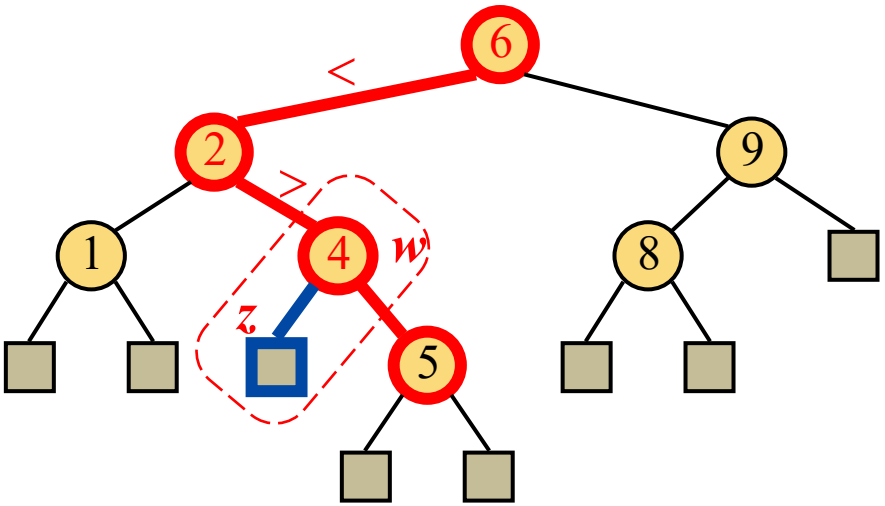
Deletion Case 1: One External Child

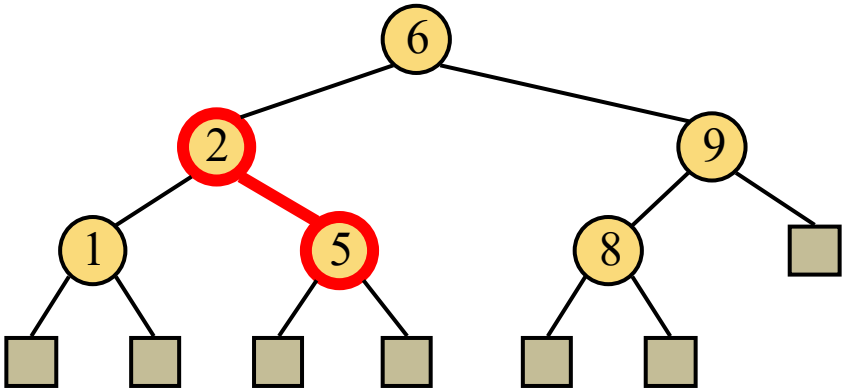
Suppose that the node w we want to remove has an external child, which we call z.

To remove w we

- remove w and z from the tree
- promote the other child of w to take w's place

This preserves the BST property



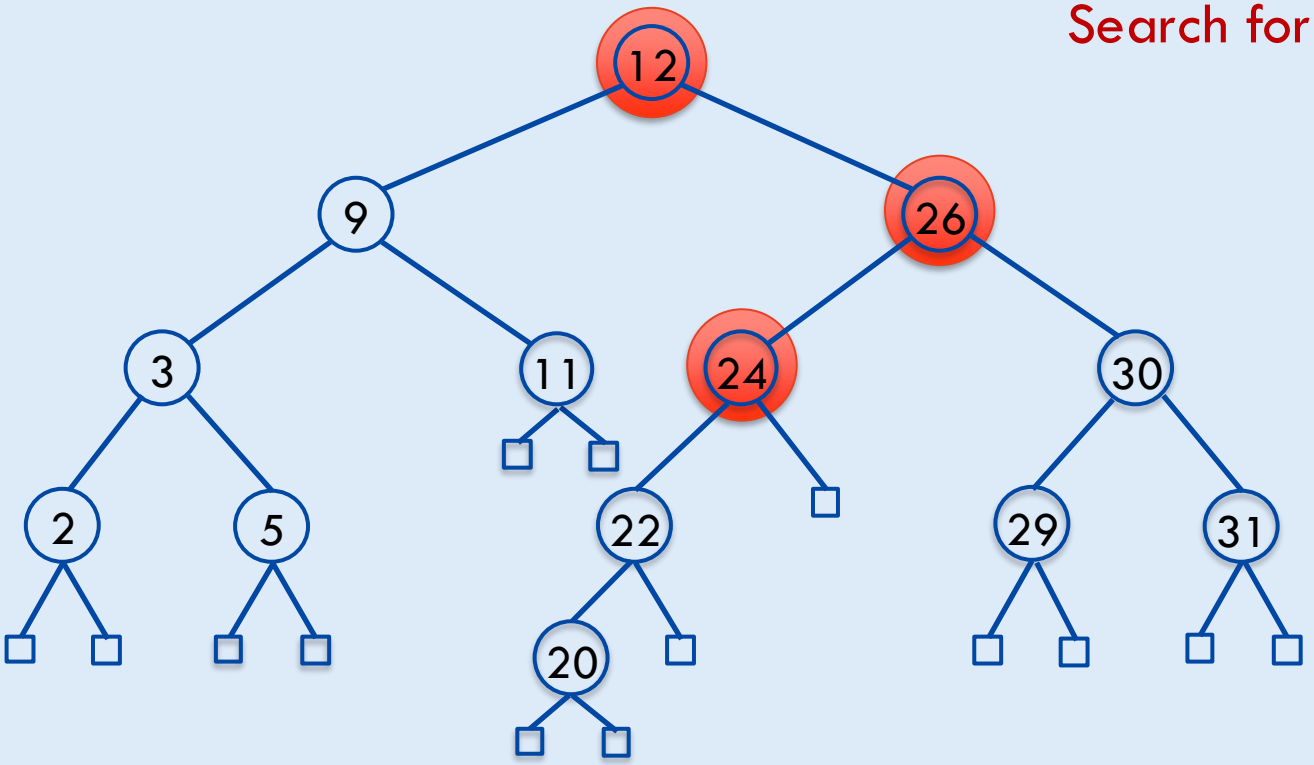


© Goodrich, Tamassia, Goldwasser

Interactive Example: Delete 24

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

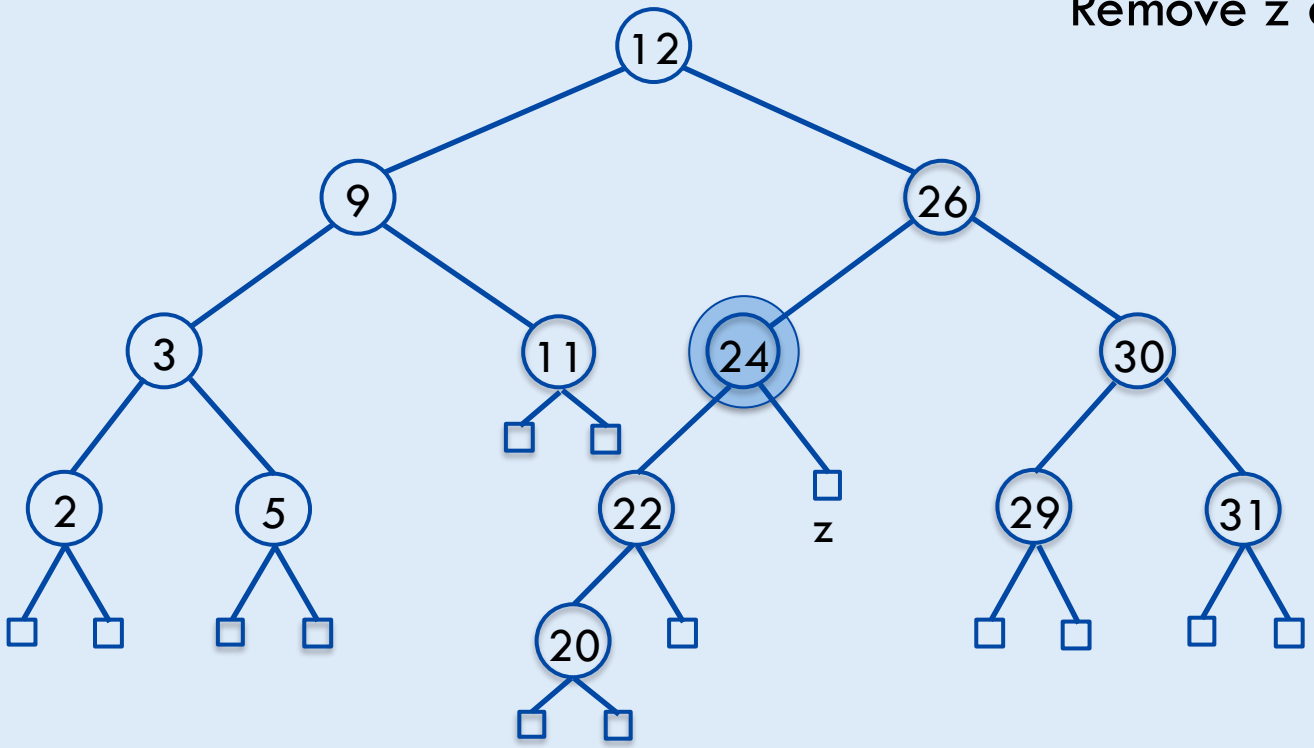
Search for 24



Interactive Example: Delete 24

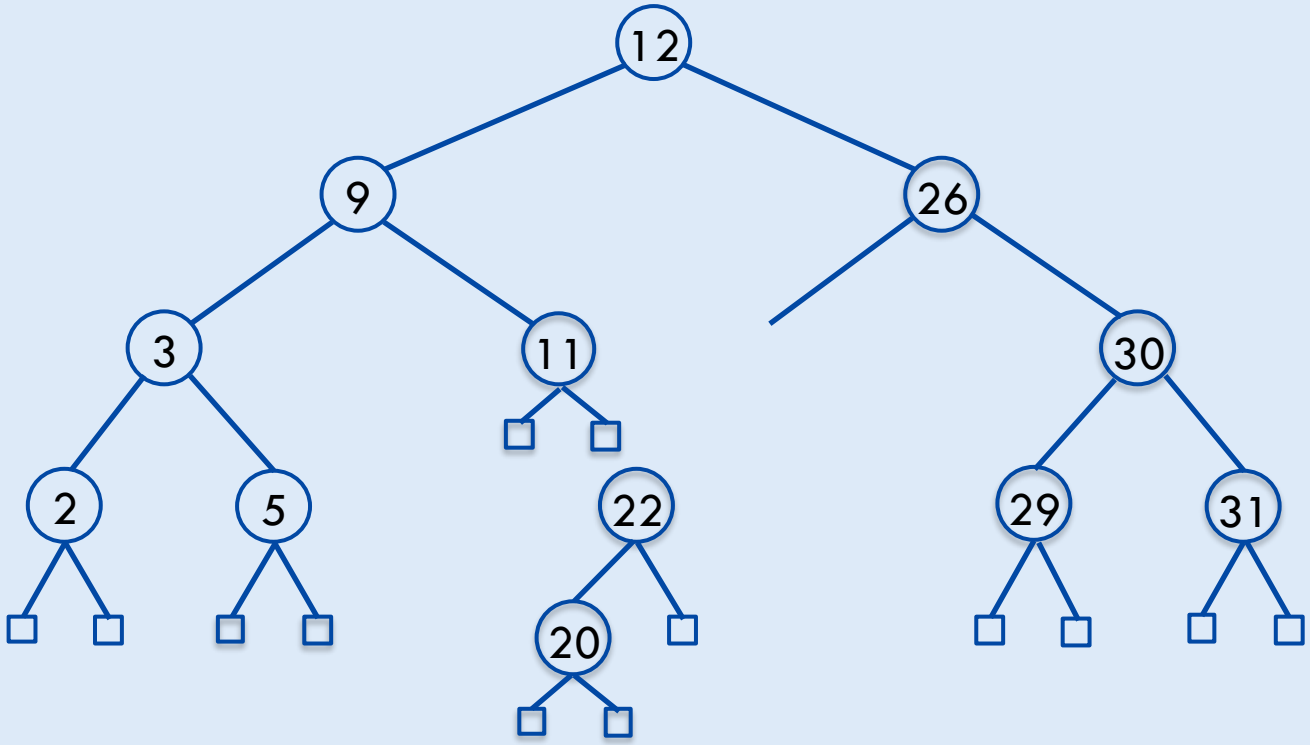
S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Remove z and w



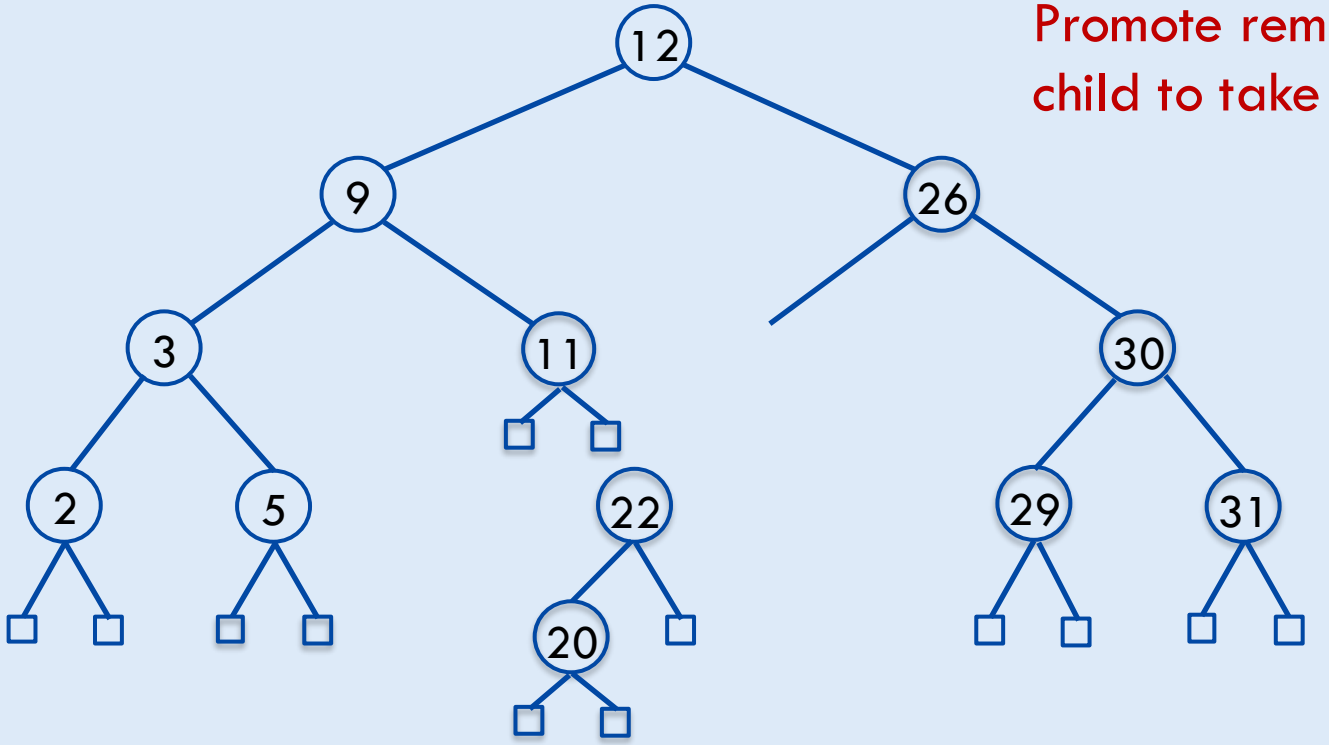
Interactive Example: Delete 24

S={2,3,5,9,11,12,20,22,24,26,29,30,31}



Interactive Example: Delete 24

S={2,3,5,9,11,12,20,22,24,26,29,30,31}



Promote remaining child to take spot of w

Deletion : Case 2 where there are 2 children

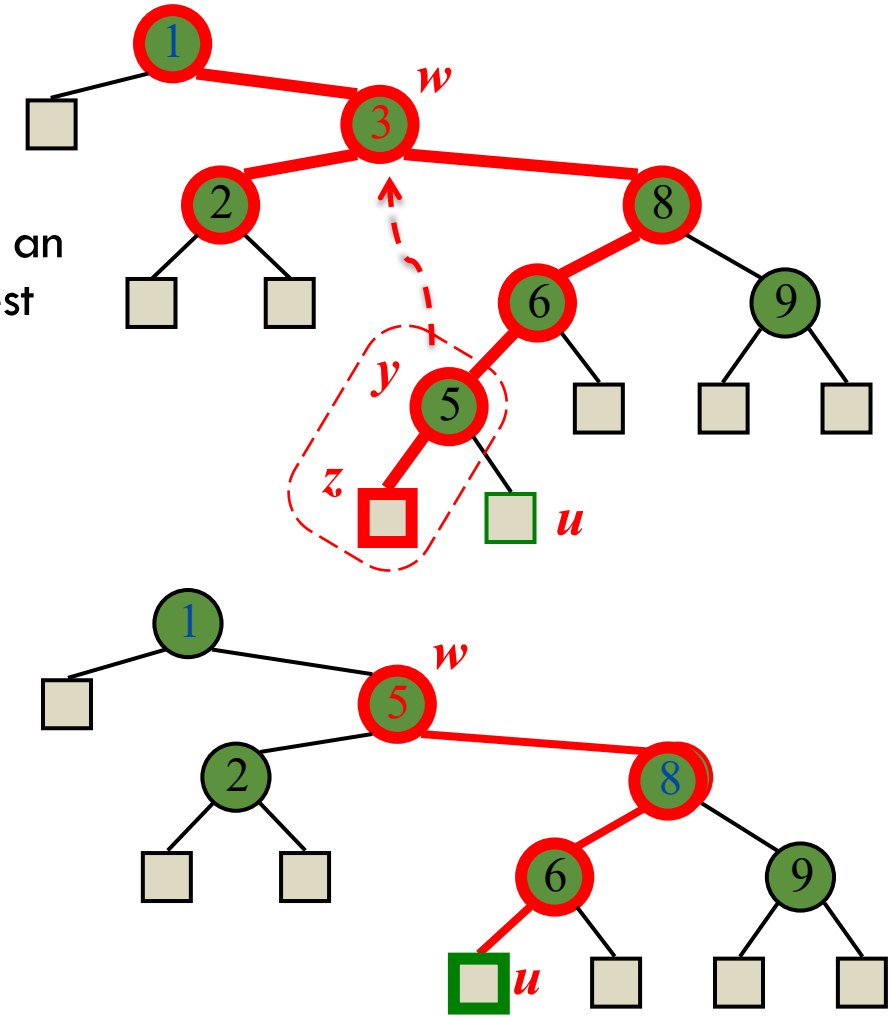
Suppose that the node w we want to remove has two internal children.

To remove w we

- find the internal node y following w in an inorder traversal (i.e., y has the smallest key among the right subtree under w)
- we copy the entry from y into node w
- we remove node y and its left child z, which must be external, using previous case

This preserves the BST property

Example: remove(3)



© Goodrich, Tamassia, Goldwasser

Deletion algorithm

```
def remove(k)
    w ← search(k, root)
    if w.isExternal() then
        # key not found
        return null
    else if w has at least one external child z then
        remove z
        promote the other child of w to take w's place
        remove w
    else
        # y is leftmost internal node in the right subtree of w
        y ← immediate successor of w
        replace contents of w with entry from y
        remove y as above
```

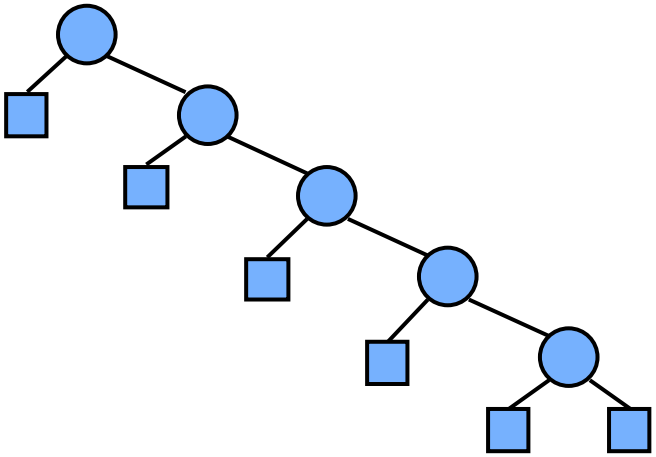
```
def remove(k, root):
    w = search(k, root)
    if w is None or (w.left is None and w.right is None):  # Key not found
        return None
    elif w.left is None or w.right is None:  # w has at least one external child
        z = w.left if w.left is not None else w.right
        w = z  # Promote the other child to take w's place
    else:
        y = find_min(w.right)  # y is the leftmost internal node in the right subtree
        w.key = y.key  # Replace contents of w with y
        remove(y.key, w.right)  # Remove y
```


© Goodrich, Tamassia, Goldwasser

Complexity

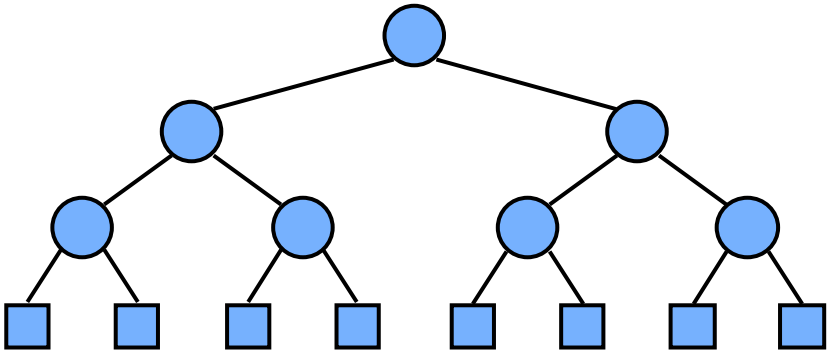
Consider a binary search tree with n items and height h:

- the space used is O(n)
- get, put and remove take O(h) time



The height h can be n in the worst case and log n in the best case.

Therefore the best one can hope is that tree operations take O(log n) time but in general we can only guarantee O(n). But the former can be achieved with better insertion routines.



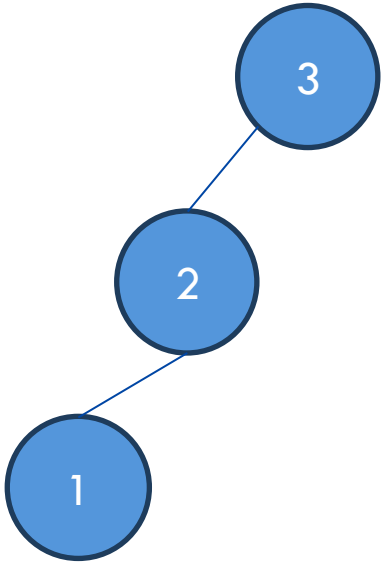
Complexity

Consider a binary search tree with n items and height h:

- the space used is O(n)
- get, put and remove take O(h) time

Look at the leaf leaning BST, what is the complexity analysis for:

- Searching
- Insertion
- Deletion



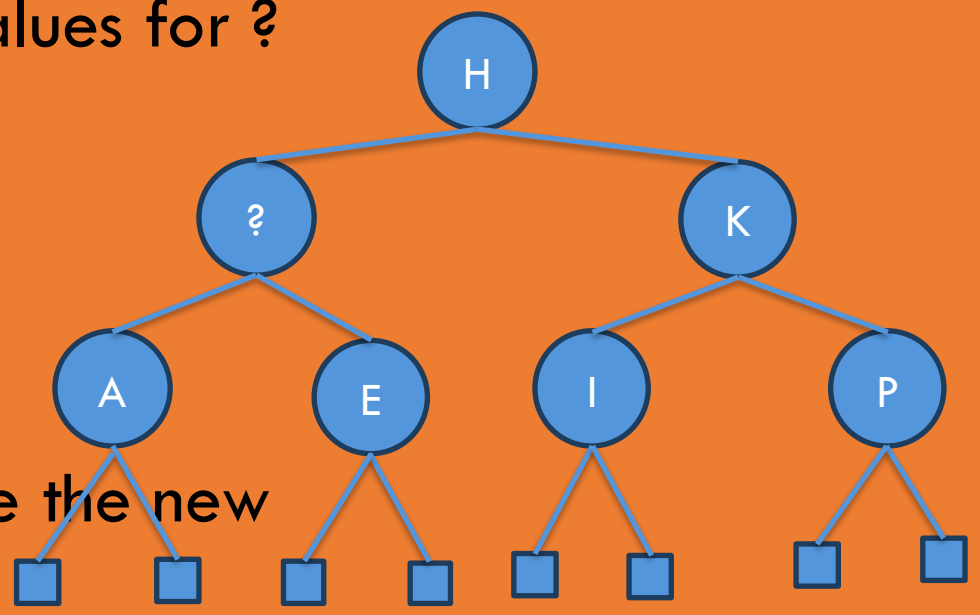
Applications

- Dictionary
- File System
- Auto-complete

Challenge 5A

Perform the following operations on this BST in order assuming A, E, I, P have 2 external nodes as children:

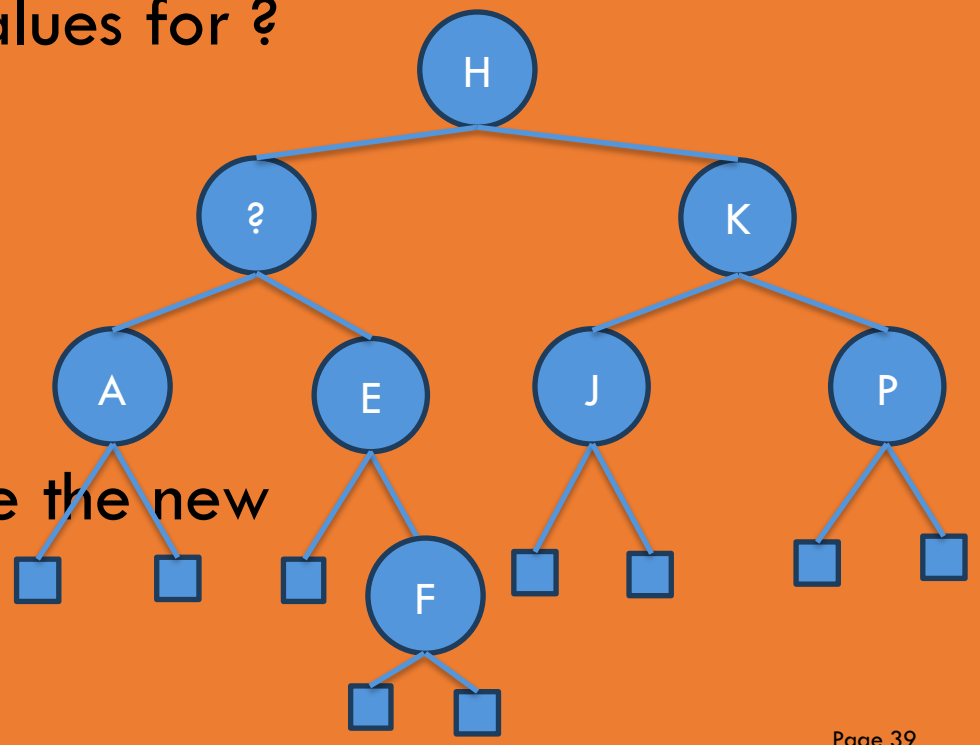
1. What is the range of values for ?
2. Insert the key 'F'
3. Delete the key 'I'
4. Insert the key 'J'
5. Delete the key 'H' (assuming ?=B) and note the new 'root' of the tree



Challenge 5A

Perform the following operations on this BST in order assuming A, E, I, P have 2 external nodes as children:

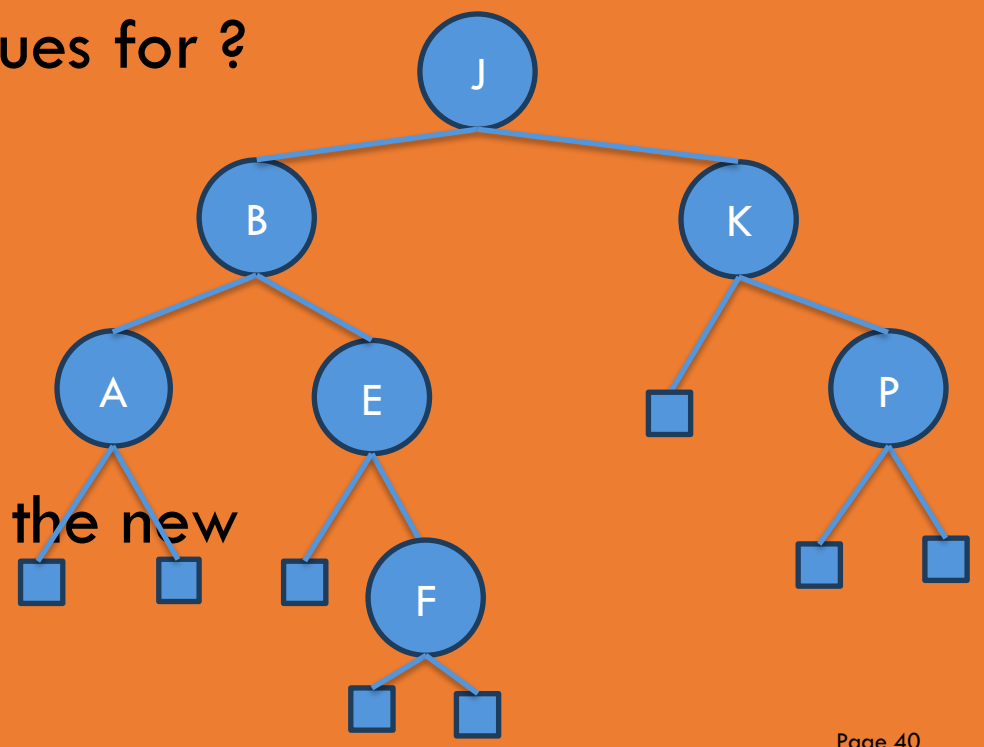
1. What is the range of values for ?
2. Insert the key 'F'
3. Delete the key 'I'
4. Insert the key 'J'
5. Delete the key 'H' (assuming ?=B) and note the new 'root' of the tree



Challenge 5A

Perform the following operations on this BST in order assuming A, E, I, P have 2 external nodes as children:

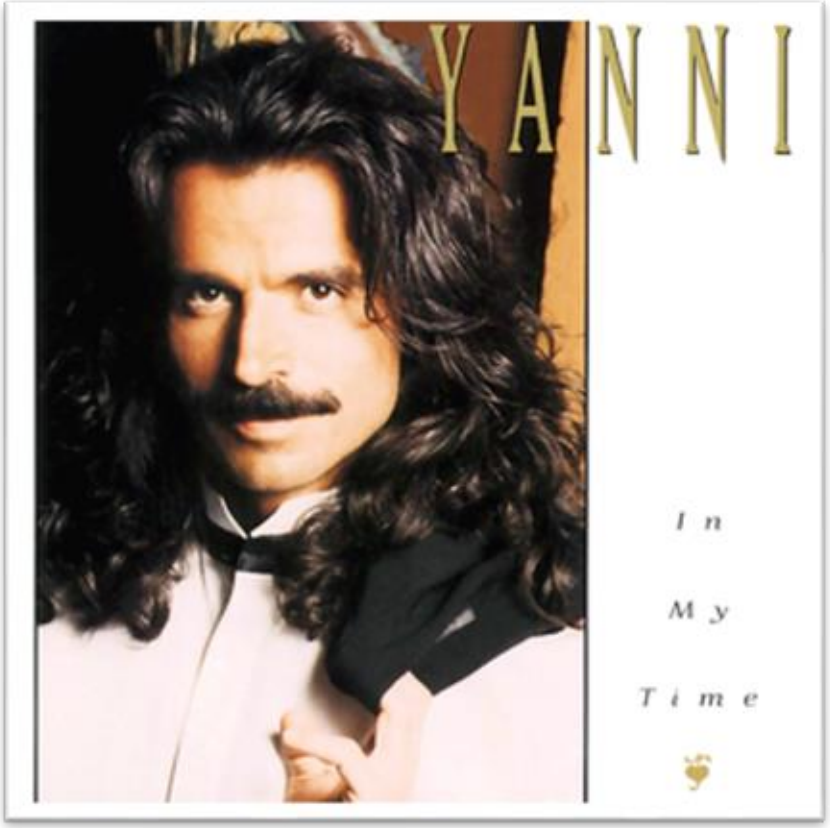
1. What is the range of values for ?
2. Insert the key 'F'
3. Delete the key 'I'
4. Insert the key 'J'
5. Delete the key 'H' (assuming ?=B) and note the new 'root' of the tree



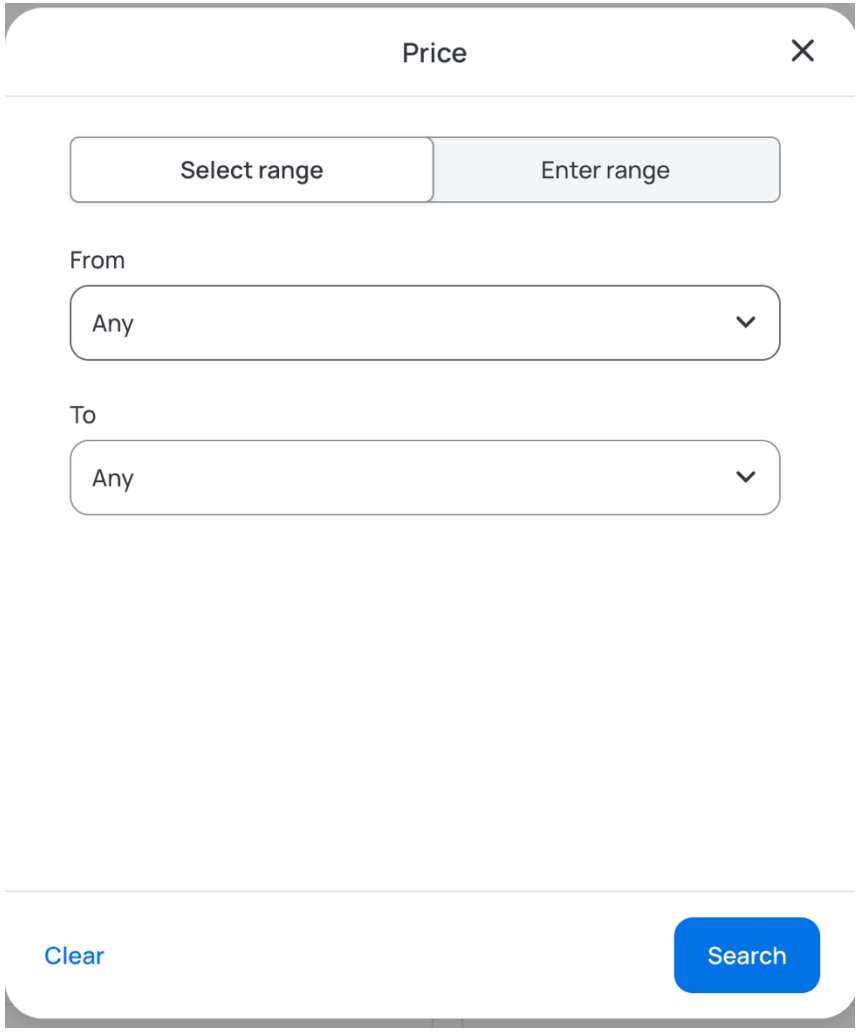
Menti

https://www.menti.com/aldb6gqiqyr7

Break!



Range Queries



https://www.carsales.com.au/cars/new-south-wales-state/sydney-region/

Range Queries

A range query is defined by two values k_1 and k_2. We are to find all keys k stored in T such that $k_1 \leq k \leq k_2$

E.g., find all cars on eBay priced between 10K and 15K.

The algorithm is a restricted version of inorder traversal. When at node v:

- if $key(v) < k_1$: Recursively search right subtree
- if $k_1 \leq key(v) \leq k_2$: Recursively search left subtree, add v to range output, search right subtree
- if $k_2 < key(v)$: Recursively search left subtree

Pseudo-code

```
def range_search(T, k1, k2)
    output ← []
    range(T.root, k1, k2)

def range(v, k1, k2)
    if v is external then
        return null
    if key(v) > k2 then
        range(v.left, k1, k2)
    else if key(v) < k1 then
        range(v.right, k1, k2)
    else
        range(v.left, k1, k2)
        output.add(v)range(v.right, k1, k2)
```

Python-code

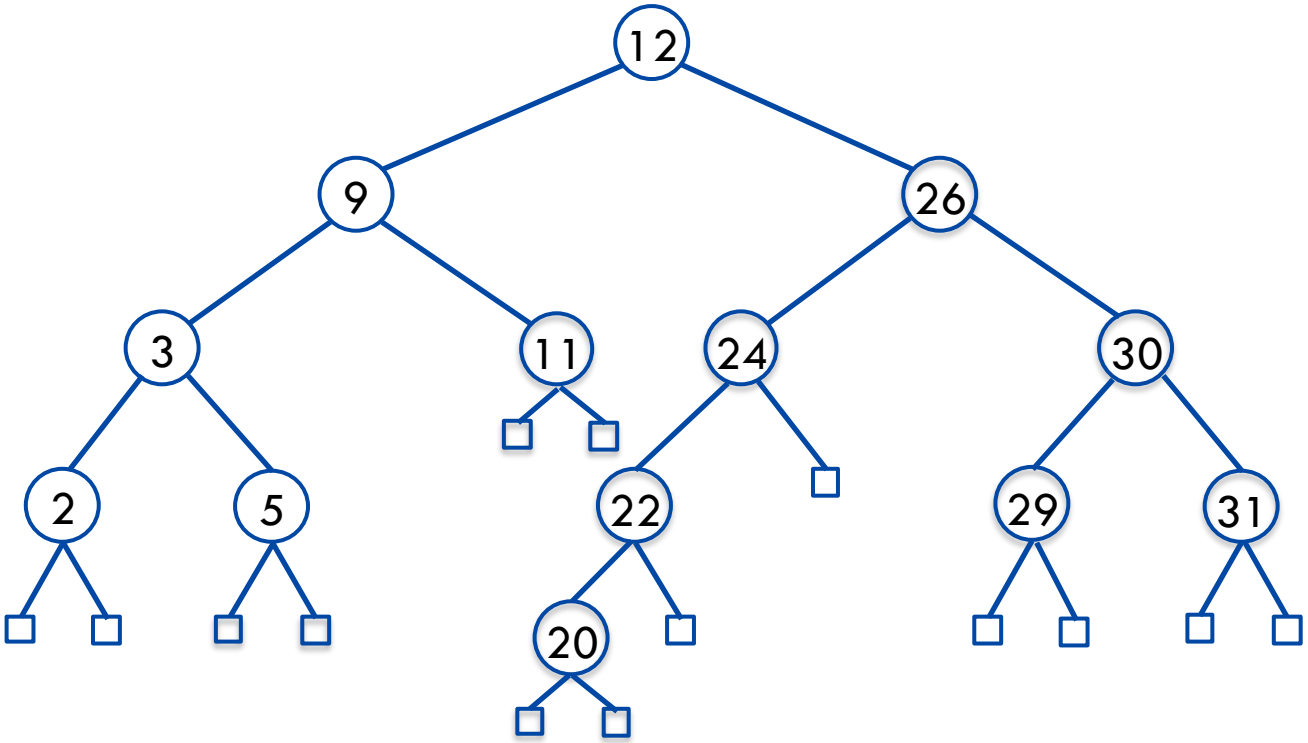
```
def range_search(T, k1, k2):
    output = []
    range (T.root, k1, k2)

def range (v, k1, k2):
    if v is None:
        return

    if v.value > k2:
        range (v.left, k1, k2)
    elif v.value < k1:
        range (v.right, k1, k2)
    else:
        range (v.left, k1, k2)
        output.append(v.value)
        range (v.right, k1, k2)
```

Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

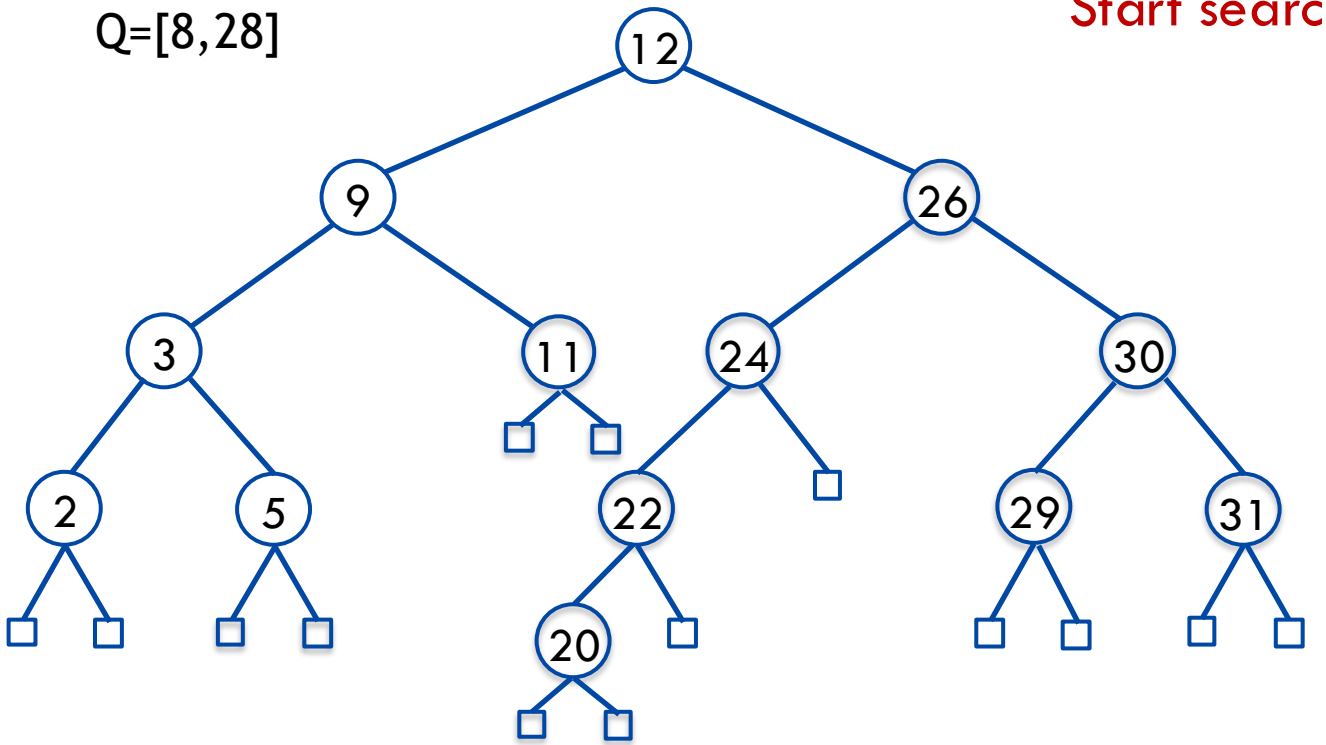


Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]

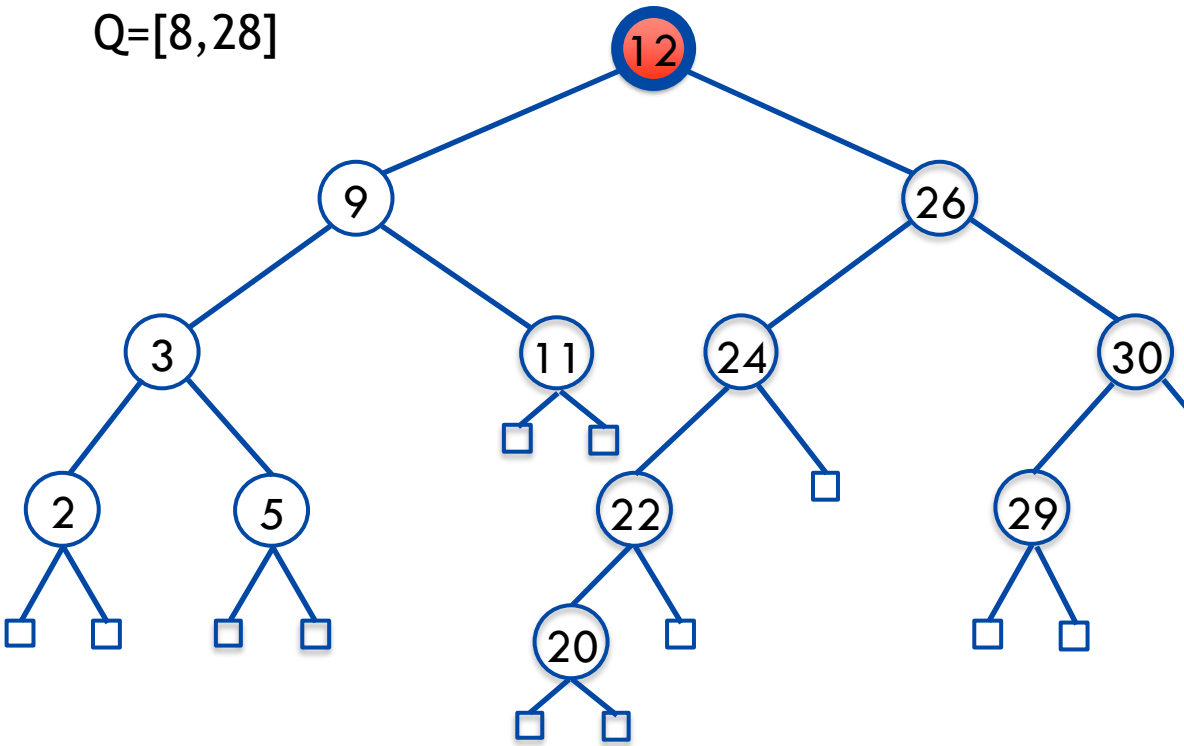
Start search



Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]



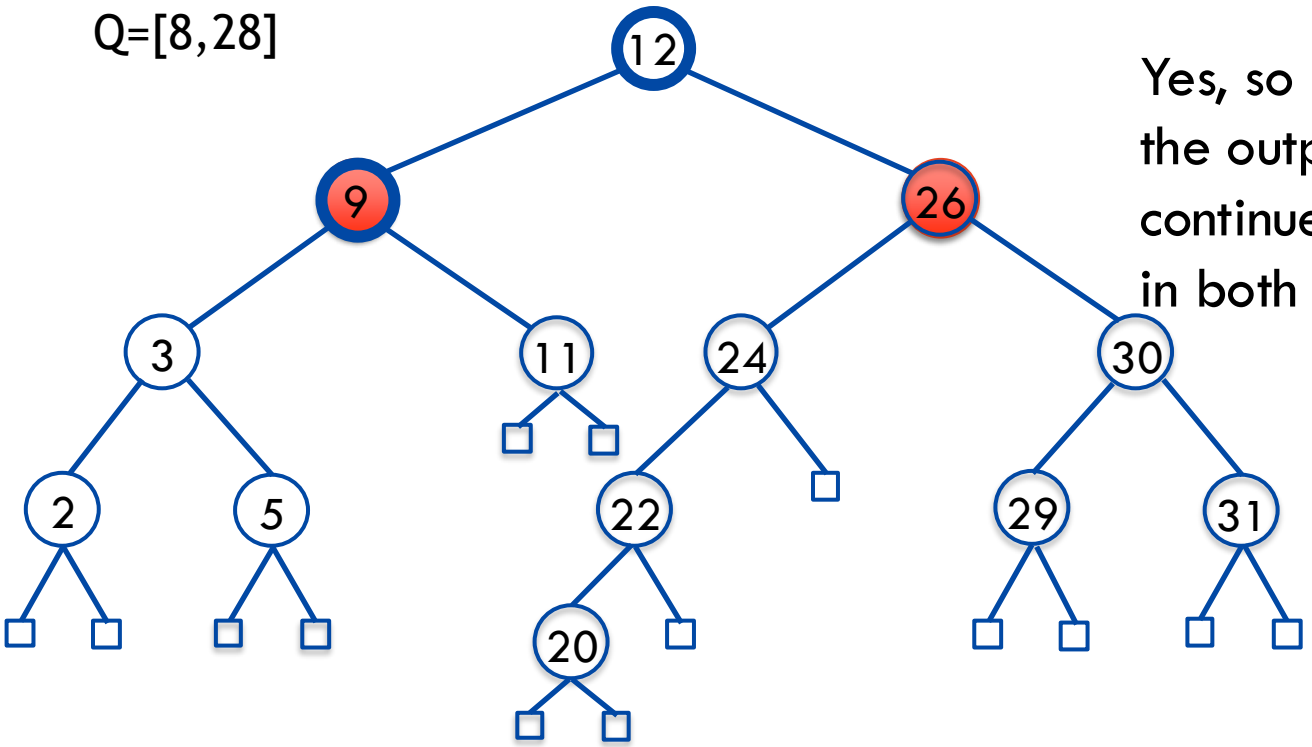
Is 12 in Q?

Yes, so 12 will be in the output and we continue the search in both subtrees

Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]



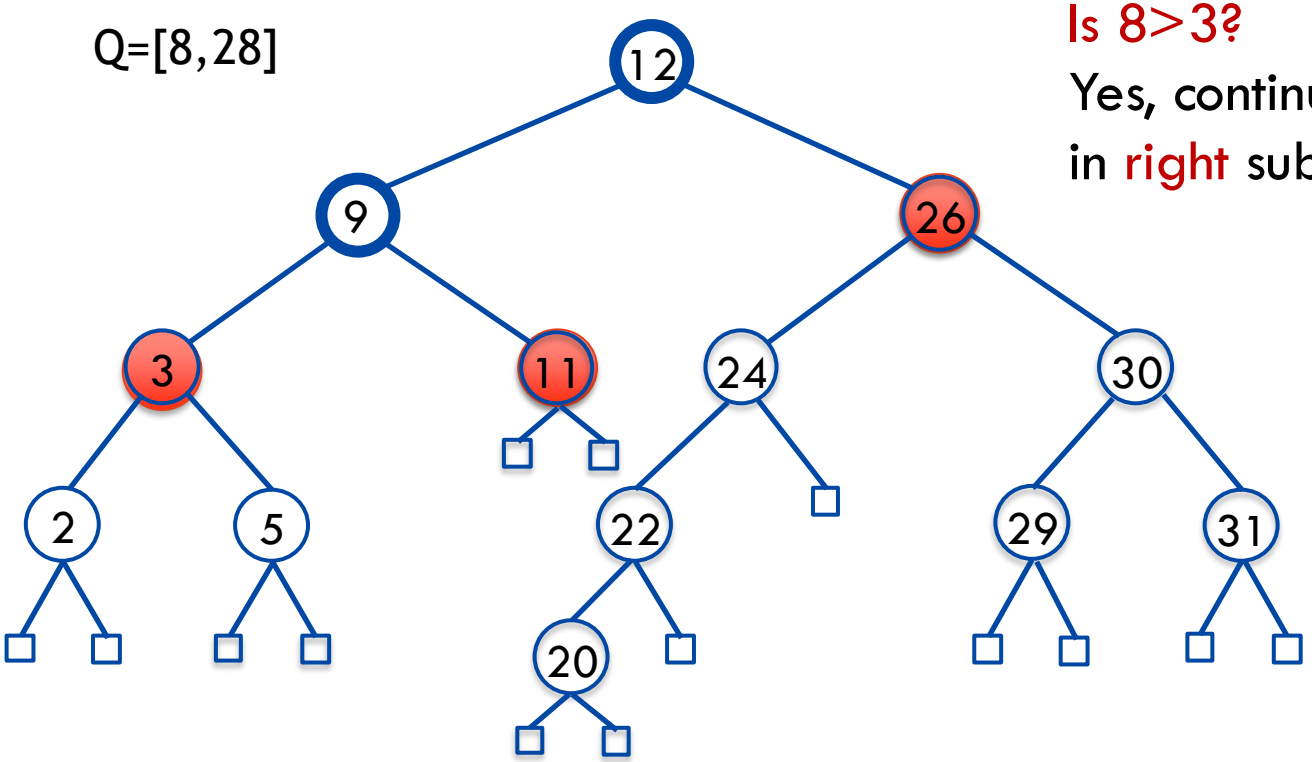
Is 9 in Q?

Yes, so 9 will be in the output and we continue the search in both subtrees

Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]



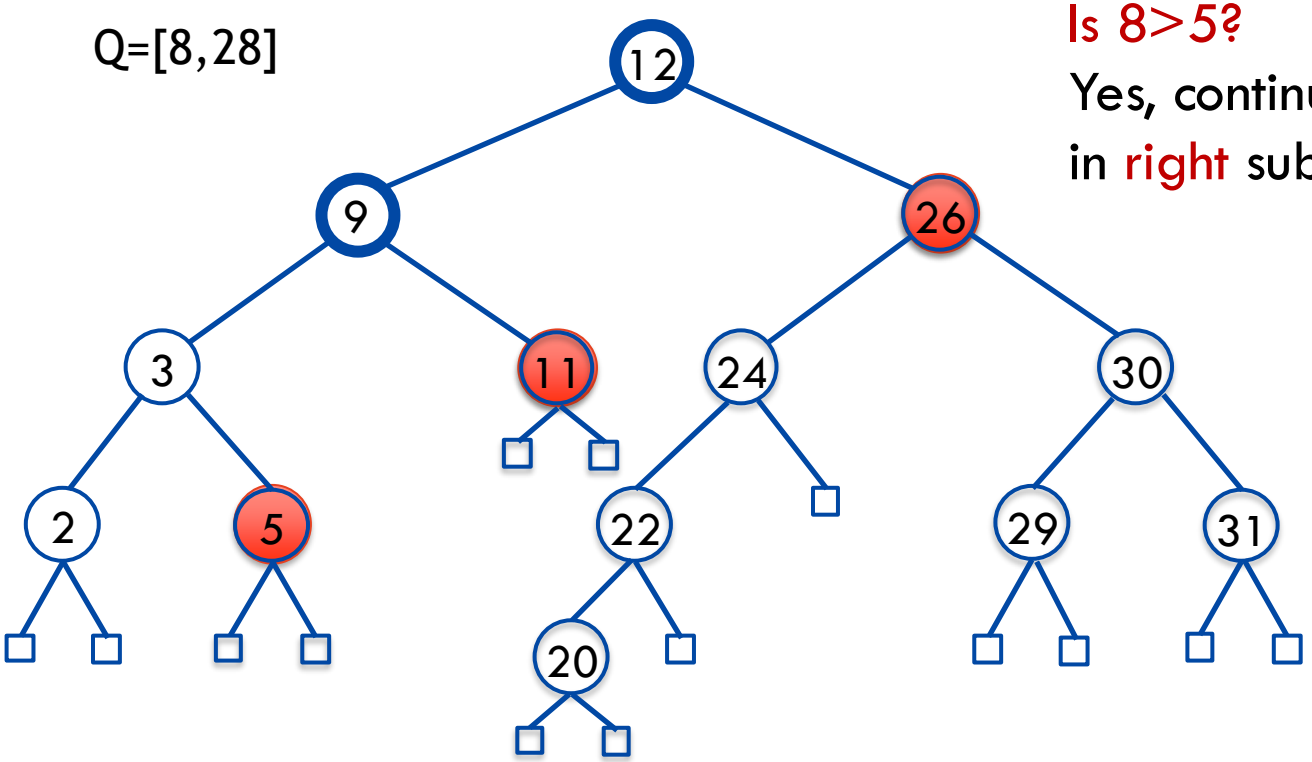
Is 8>3?

Yes, continue search in right subtree.

Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]



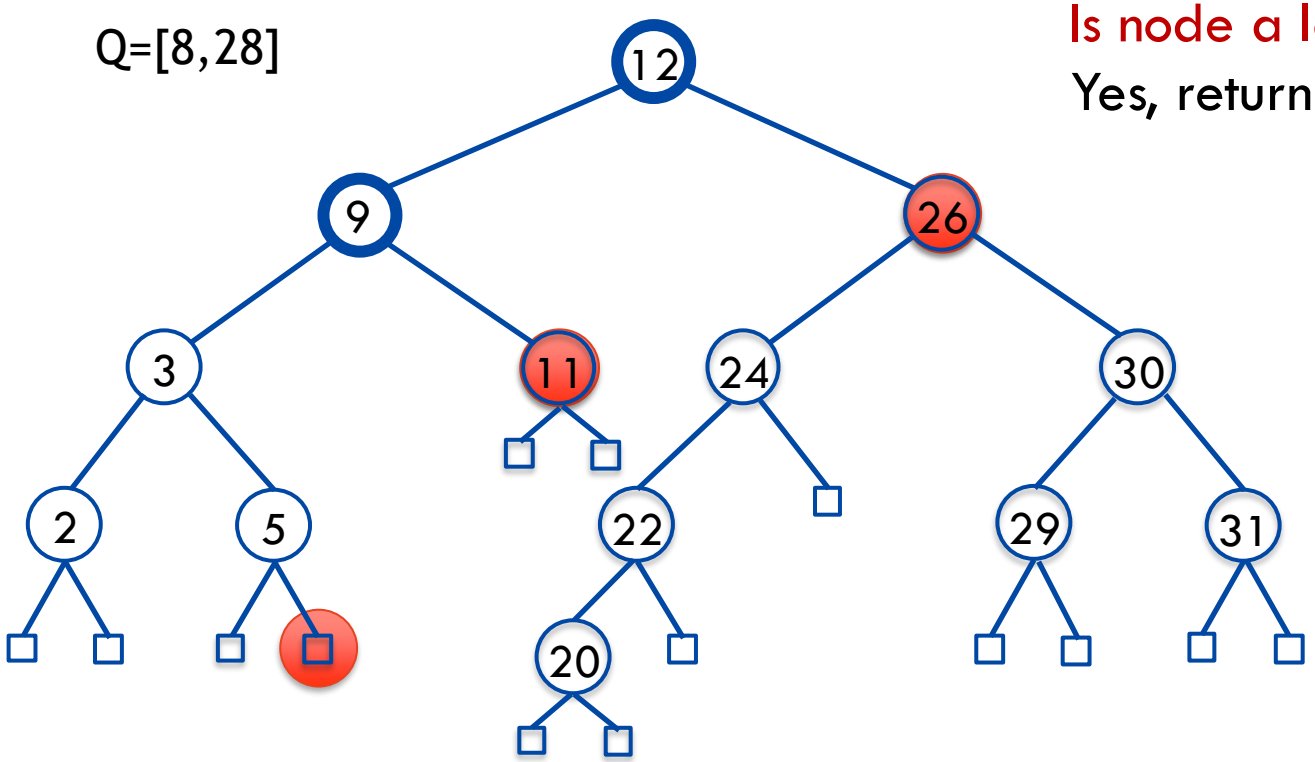
Is 8>5?

Yes, continue search in right subtree.

Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]



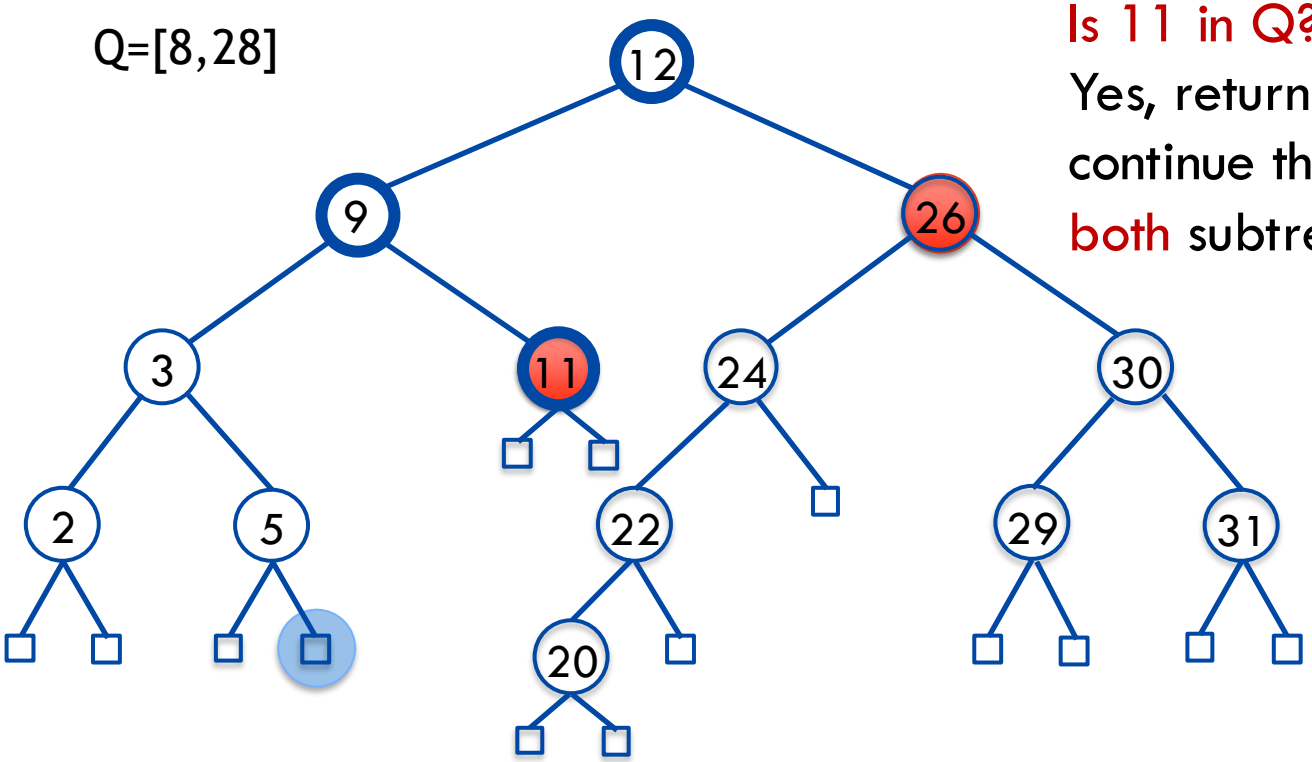
Is node a leaf?

Yes, return ∅.

Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]



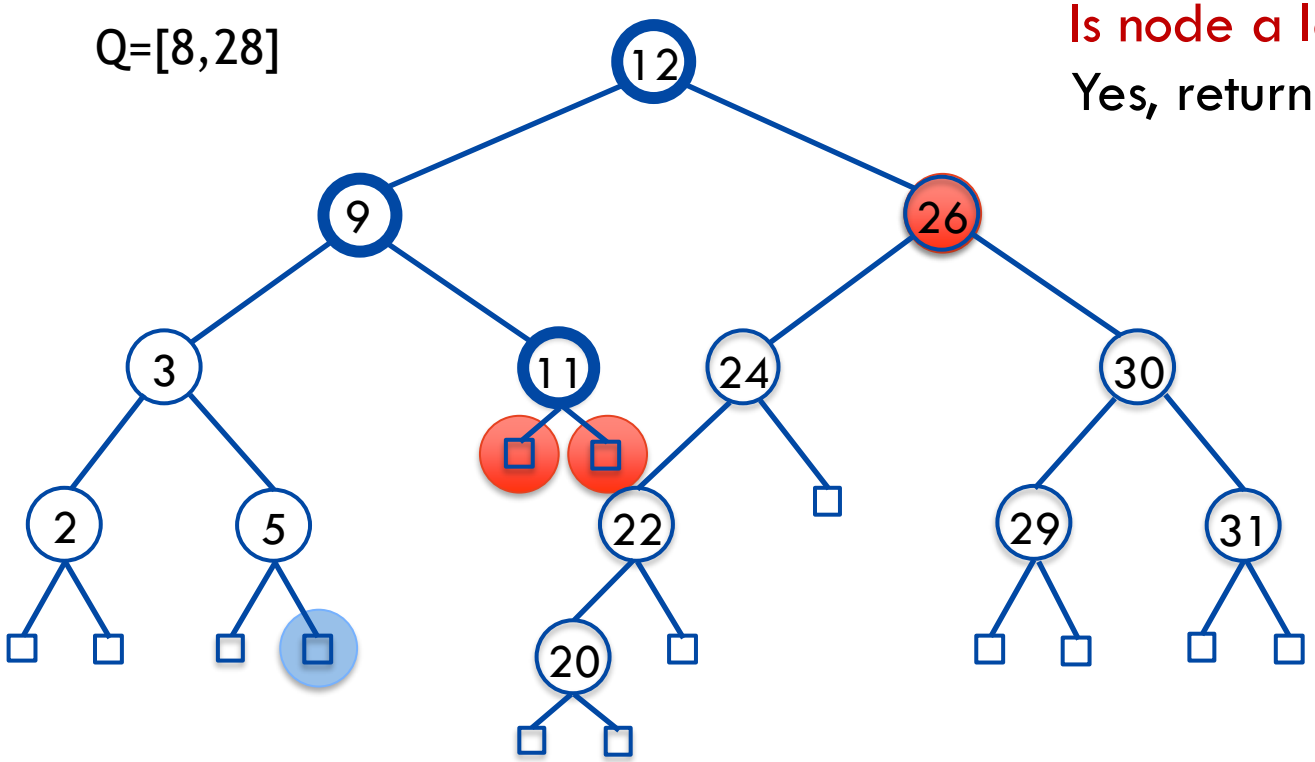
Is 11 in Q?

Yes, return 11 and continue the search in both subtrees.

Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]



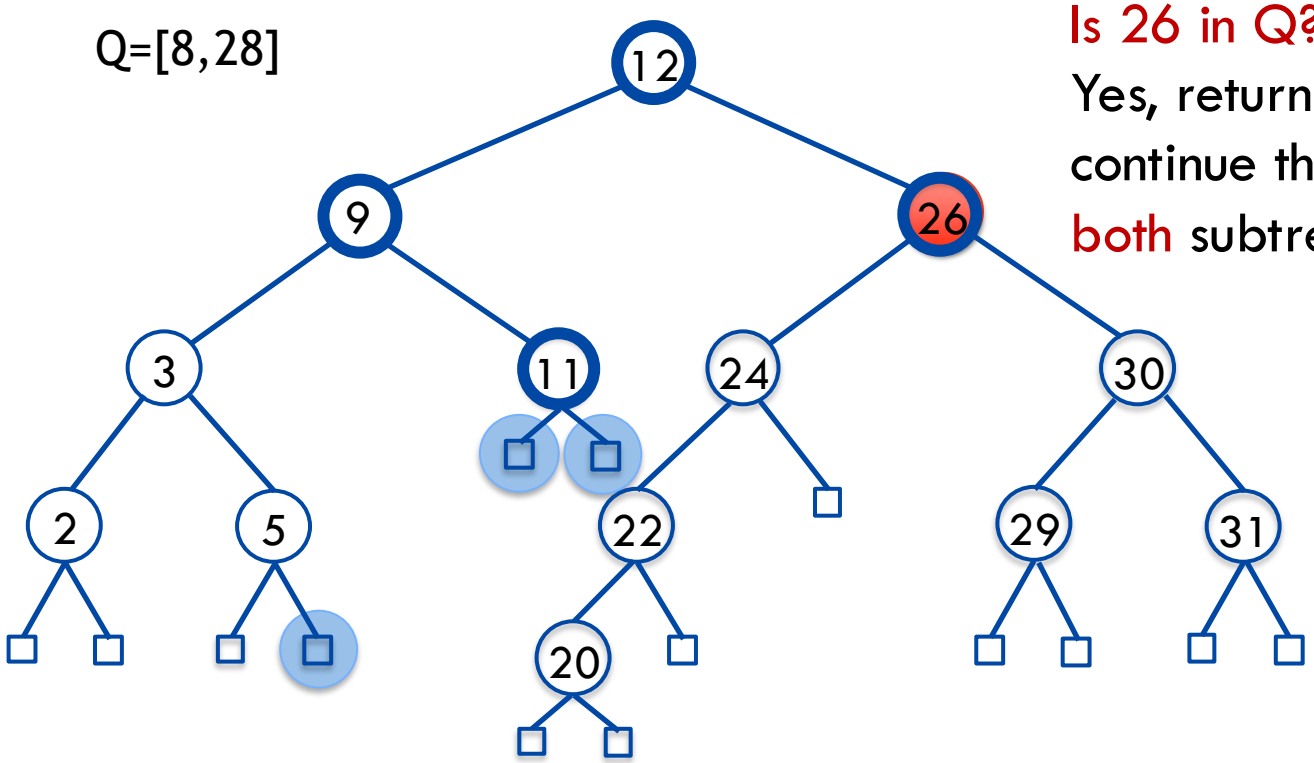
Is node a leaf (×2)?

Yes, return ∅.

Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]



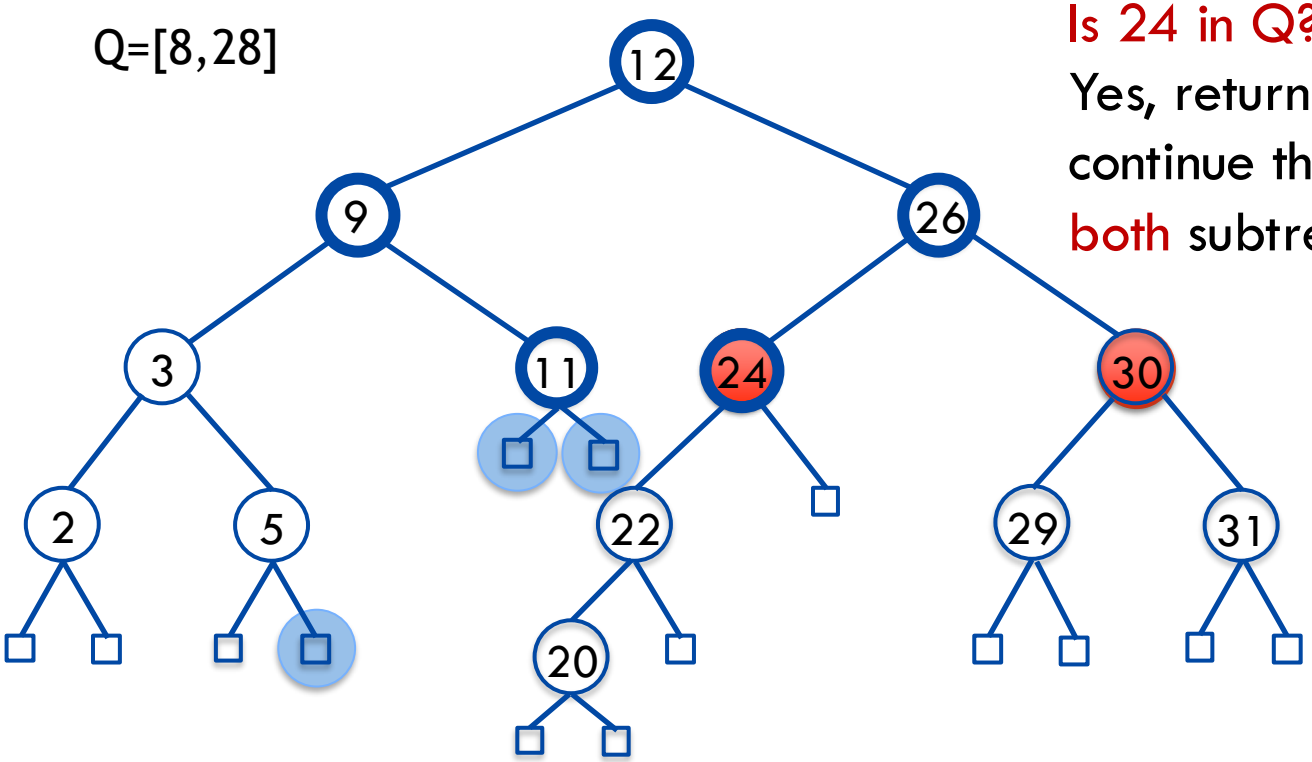
Is 26 in Q?

Yes, return 26 and continue the search in both subtrees.

Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]



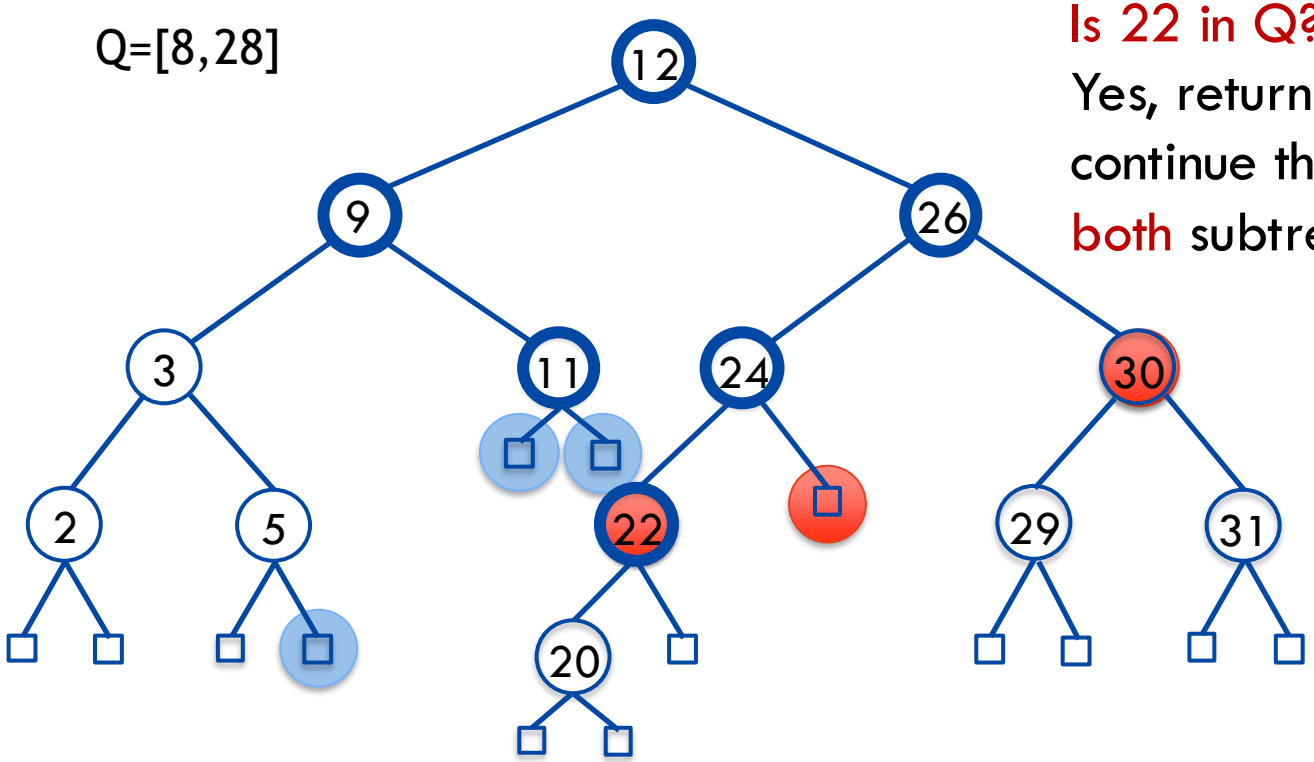
Is 24 in Q?

Yes, return 24 and continue the search in both subtrees.

Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]



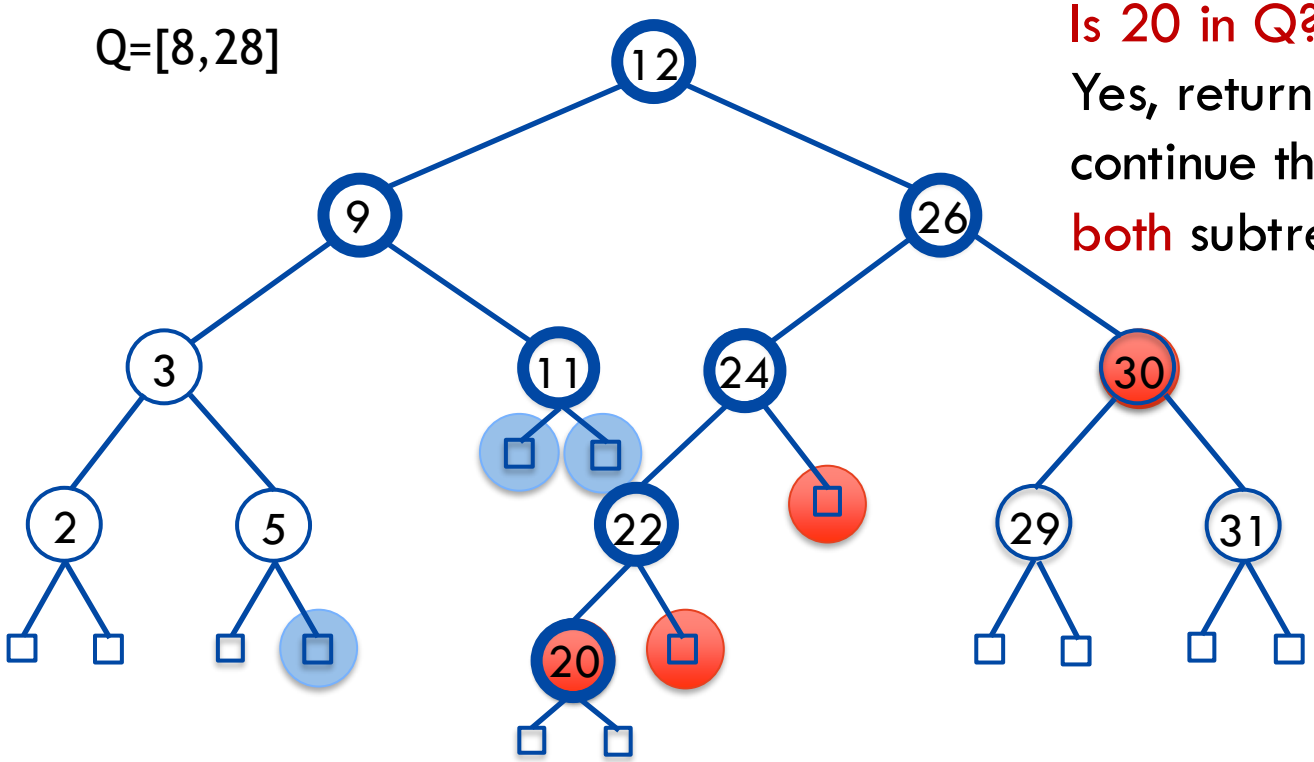
Is 22 in Q?

Yes, return 22 and continue the search in both subtrees.

Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]



Is 20 in Q?

Yes, return 20 and continue the search in both subtrees.

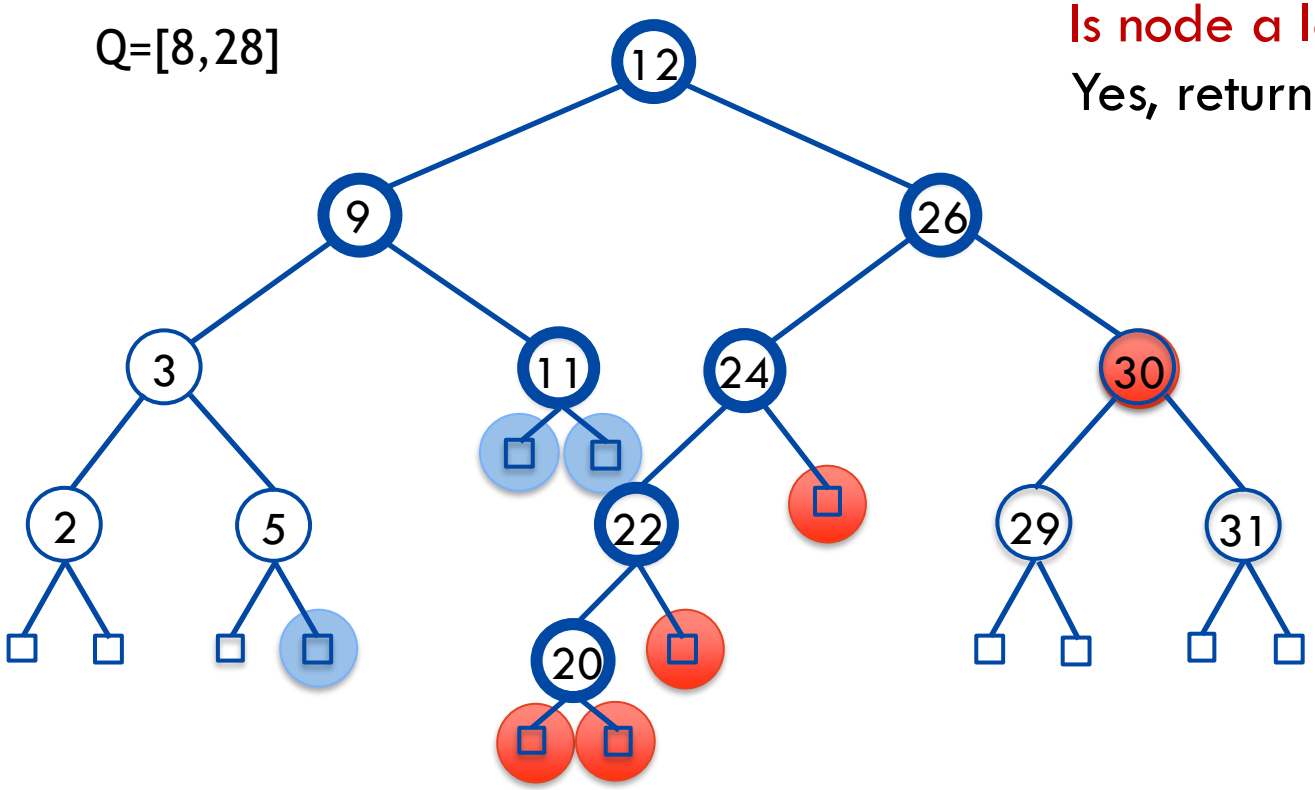
Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]

Is node a leaf (×4)?

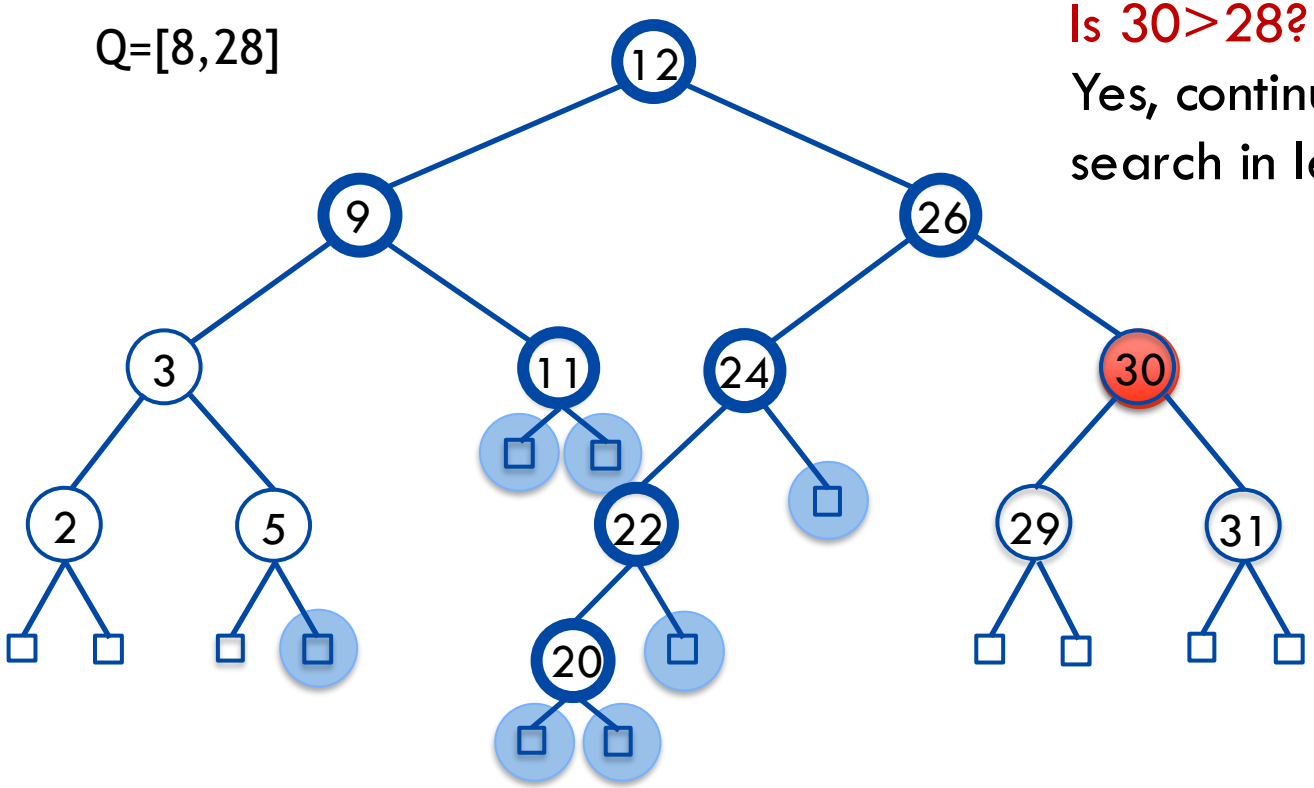
Yes, return ∅.



Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]



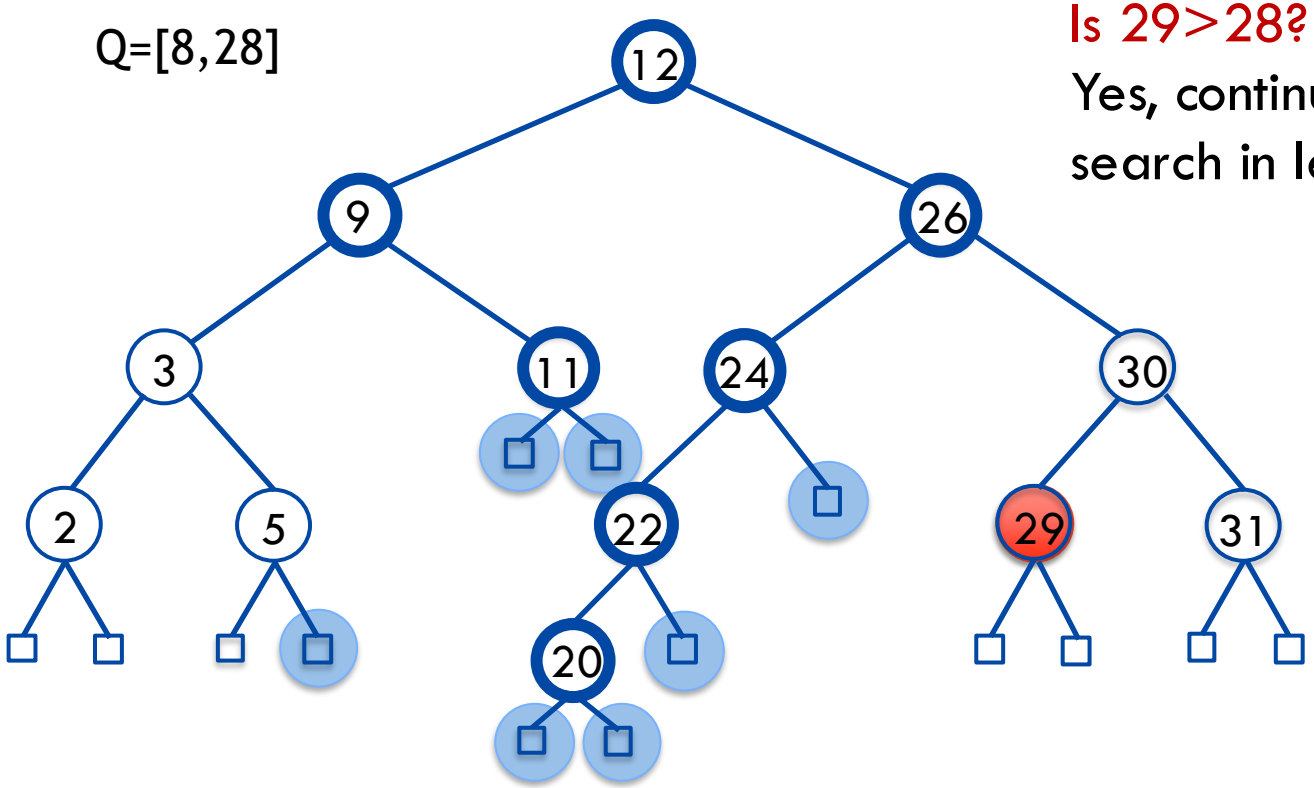
Is 30>28?

Yes, continue the search in left subtree.

Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]



Is 29>28?

Yes, continue the search in left subtree.

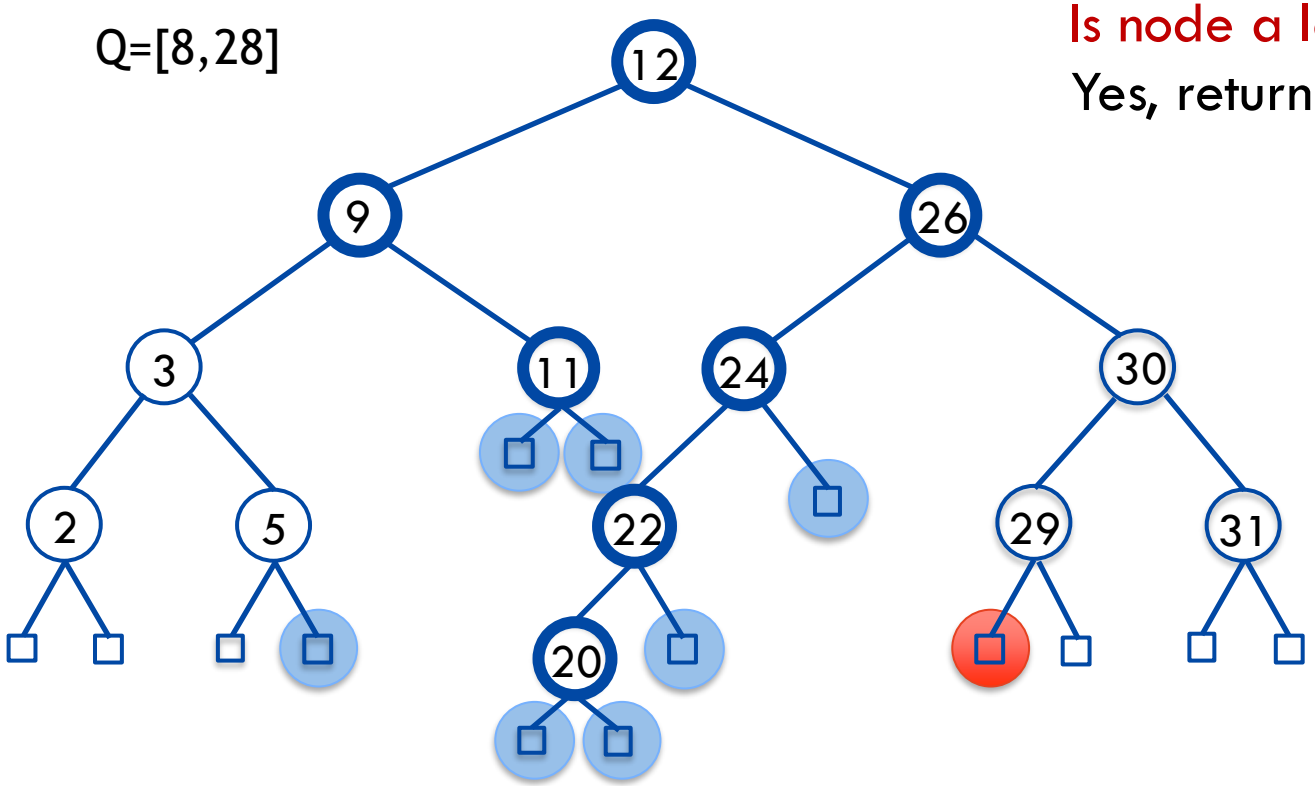
Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]

Is node a leaf?

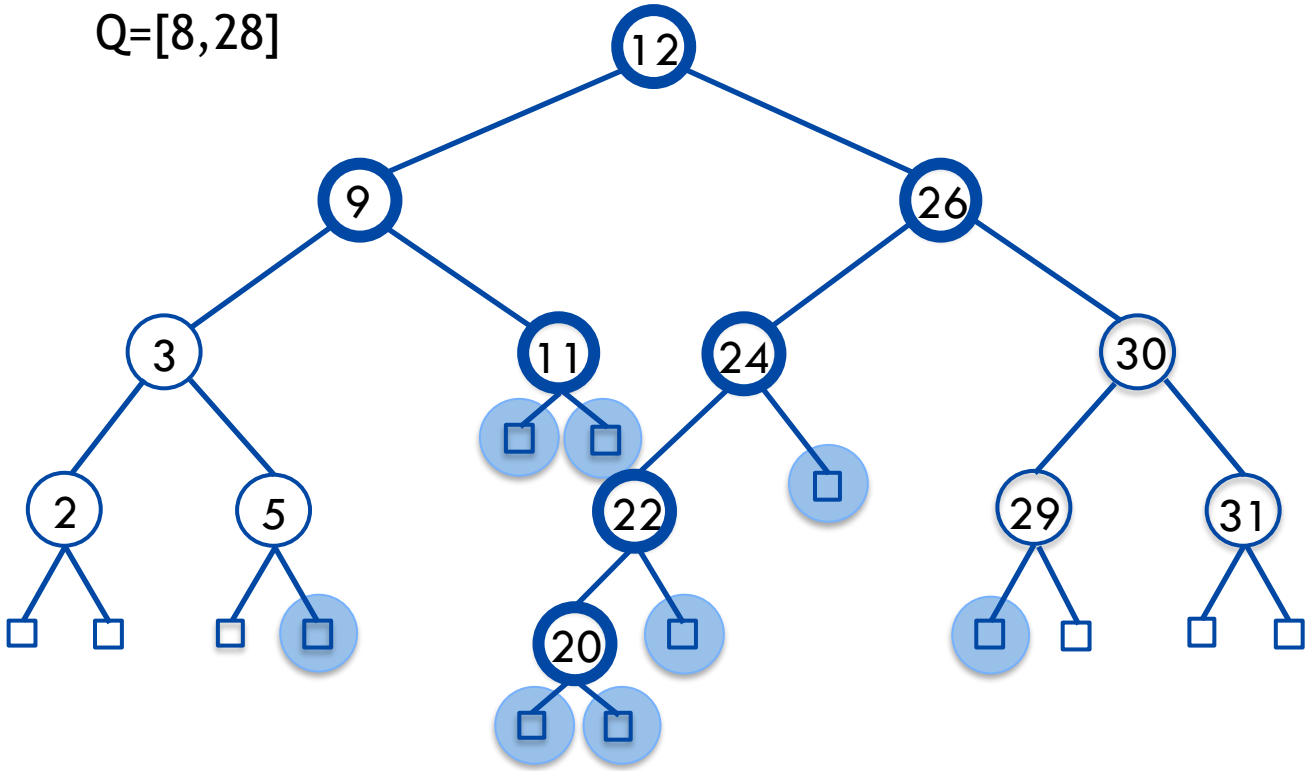
Yes, return ∅.



Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]

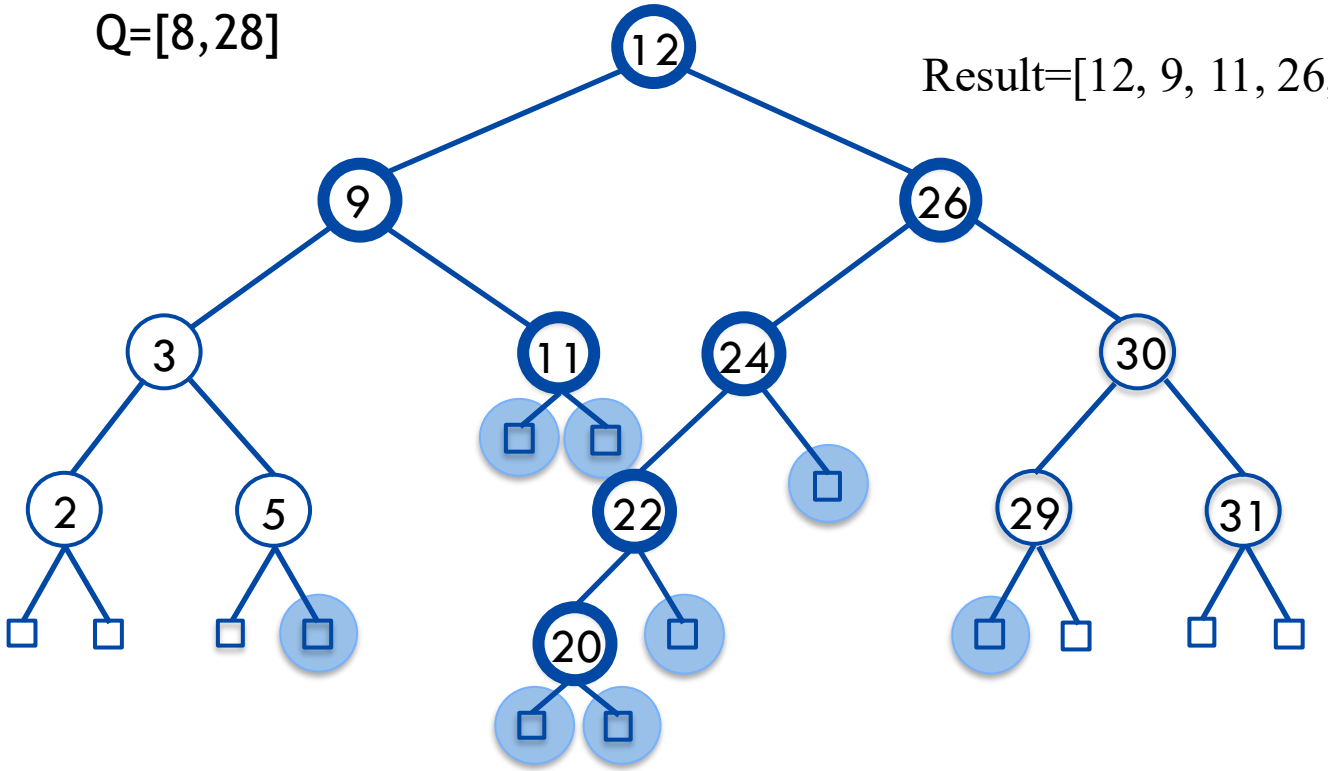


Range queries

S={2,3,5,9,11,12,20,22,24,26,29,30,31}

Q=[8,28]

Result=[12, 9, 11, 26, 24, 22, 20]



Performance

Let P_1 and P_2 be the binary search paths to k_1 and k_2
We say a node v is a:

- boundary node if v in P_1 or P_2
- inside node if key(v) in $[k_1, k_2]$ but not in P_1 or P_2
- outside node if key(v) not in $[k_1, k_2]$ but not in P_1 or P_2

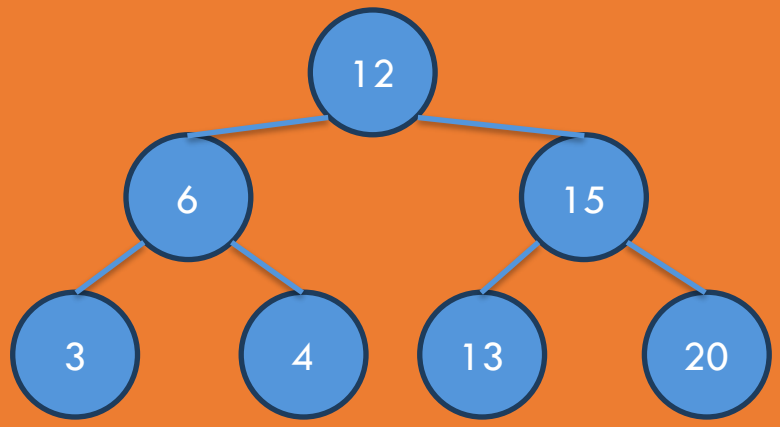
The algorithm only visits boundary and inside nodes and

- $|\text{inside nodes}| \leq |\text{output}|$
- $|\text{boundary node}| \leq 2 * \text{tree height}$

Therefore, since we only spend $O(1)$ time per node we visit, the total running time of range search is $O(|\text{output}| + \text{tree height})$

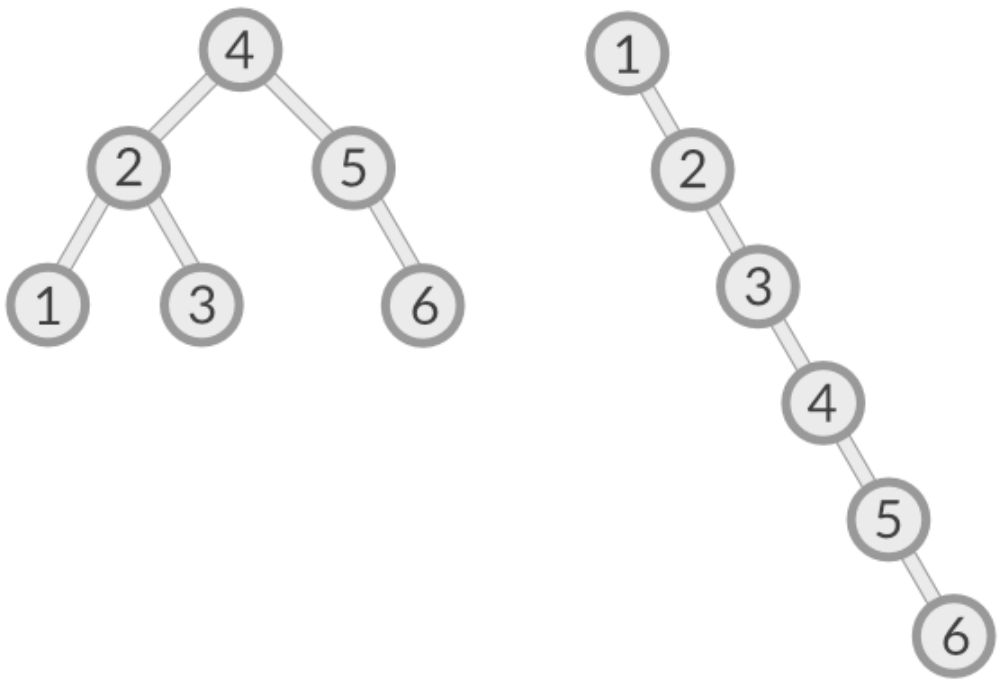
Challenge 5B

1. Write down at least 3 examples of websites or apps that use range queries.
2. (Trick Question) Given the BST below of city populations (20=20,000), what is the result of the range search query Q = (4,15].



Re-Balanced Tree

Which is better?



Credit to: https://stackoverflow.com/questions/59206128/balanced-vs-unbalanced-binary-tree-clarification-needed

Rank-balanced Trees

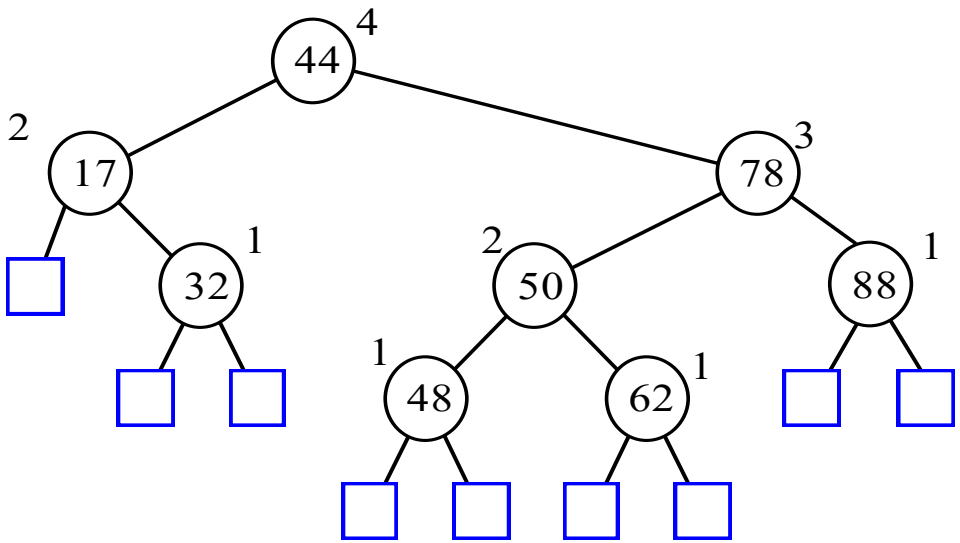
A family of balanced BST implementations that use the idea of keeping a "rank" for every node, where $r(v)$ acts as a proxy measure of the size of the subtree rooted at v

Rank-balanced trees aim to reduce the discrepancy between the ranks of the left and right subtrees:

- AVL Trees (now)
- Red-Black Trees (book)

AVL Tree Definition

AVL trees are rank-balanced trees, where r(v) is its height of the subtree rooted at v



Balance constraint: The ranks of the two children of every internal node differ by at most 1.

Fact: The height of an AVL tree storing n keys is O(log n).

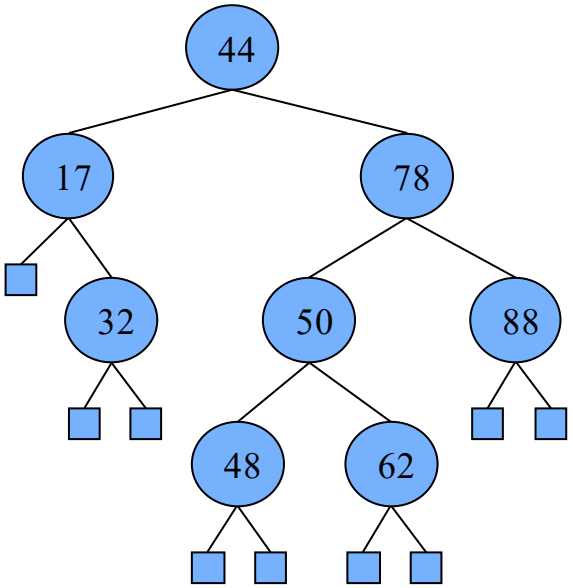
Insertion in AVL trees

Suppose we are to insert a key k into our tree:

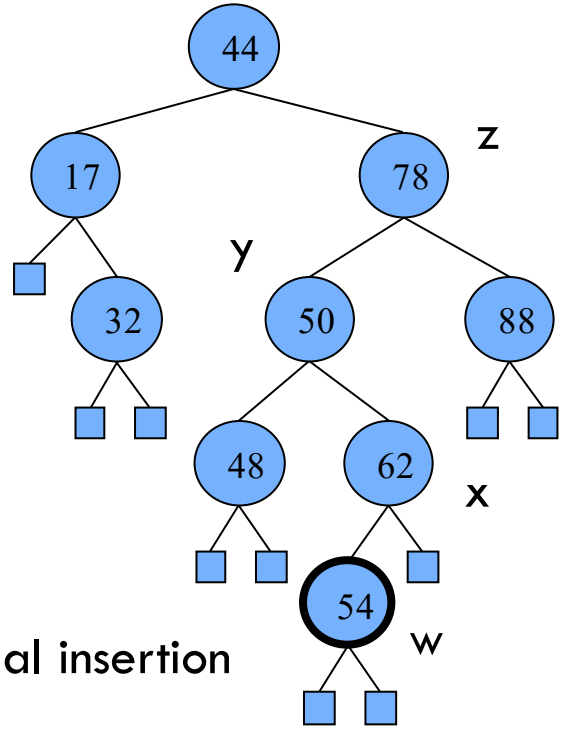
1. If k is in the tree, search for k ends at node holding k
   There is nothing to do so tree structure does not change
2. If k is not in the tree, search for k ends at external node w.
   Make this be a new internal node containing key k
3. The new tree has BST property, but it may not have AVL balance property at some ancestor of w since
   - some ancestors of w may have increased their height by 1
   - every node that is not an ancestor of w hasn't changed its height
4. We use **rotations** to re-arrange tree to re-establish AVL property, while keeping BST property

Re-establishing AVL property

- Let w be location of newly inserted node
- Let z be *lowest* ancestor of w, whose children heights differ by 2
- Let y be the child of z that is ancestor of w (taller child of z)
- Let x be child of y that is ancestor of w

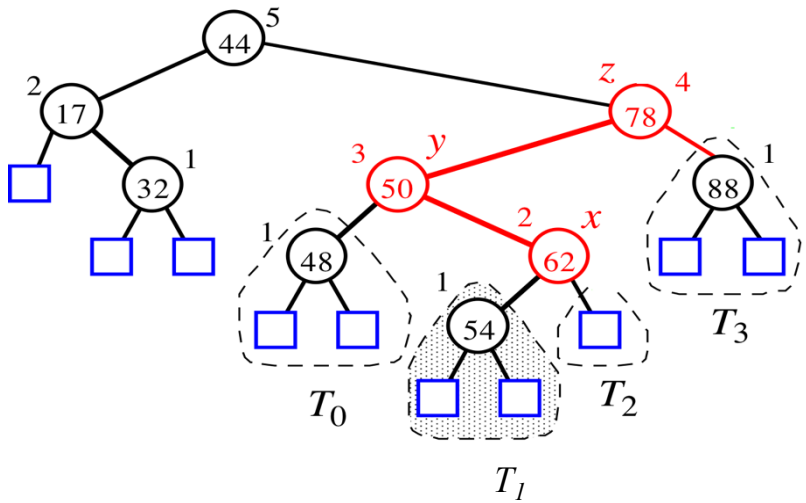


before inserting 54

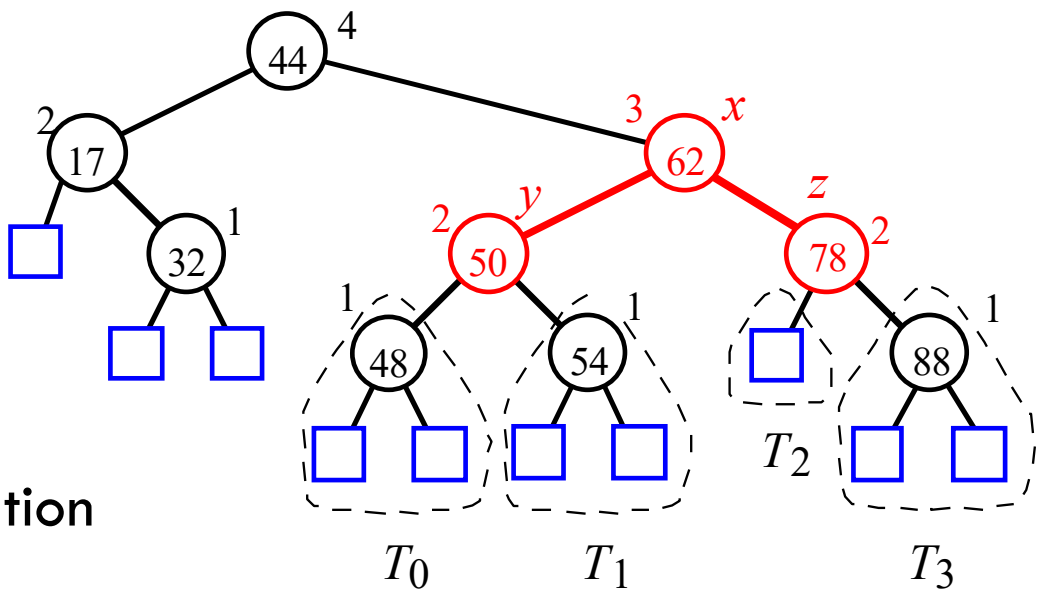


after initial insertion

Re-establishing AVL property



If tree does not have AVL property, do a trinode restructure at x, y, z



It can be argued that tree has AVL property after operation

Augmenting BST with a height attribute

But how do we know the height of each node? If we had to compute this from scratch it would take O(n) time

Therefore, we need to have this pre-computed and update the height value after each insertion and rebalancing operation:

- After we create a node w, we should set its height to be 1, and then update the height of its ancestors.
- After we rotate (z, y, x) we should update their height and that of their ancestors.

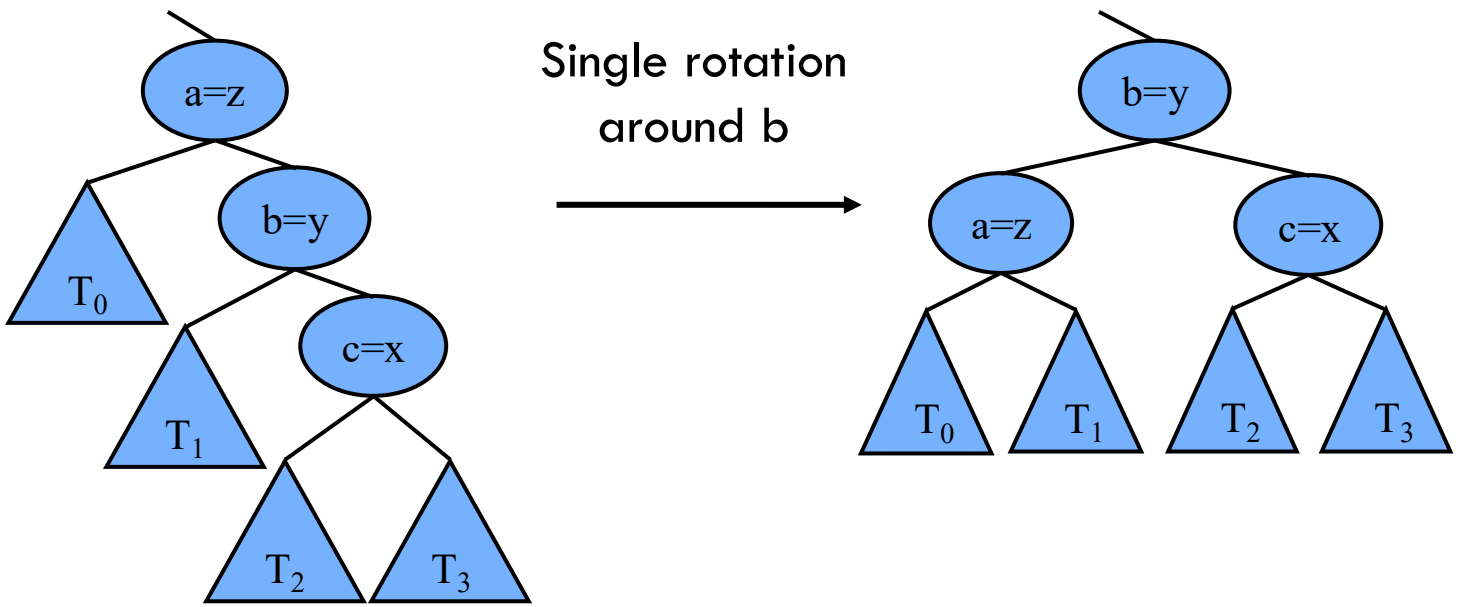
Thus, we can maintain the height only using O(h) work per insert

Improving Balance: Trinode Restructuring

Let x, y, z be nodes such that x is a child of y and y is a child of z.

Let a, b, c be the inorder listing of x, y, z

Perform the rotations so as to make b the topmost node of the three.

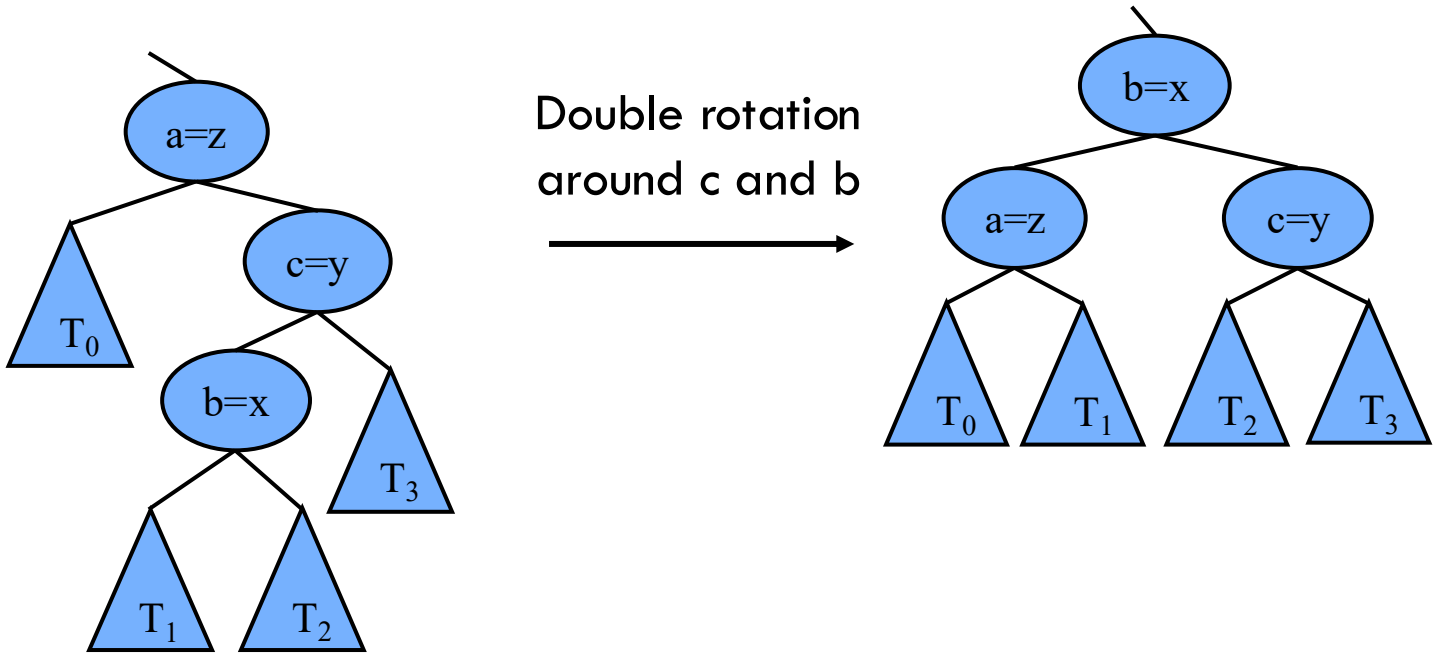


Improving Balance: Trinode Restructuring

Let x, y, z be nodes such that x is a child of y and y is a child of z.

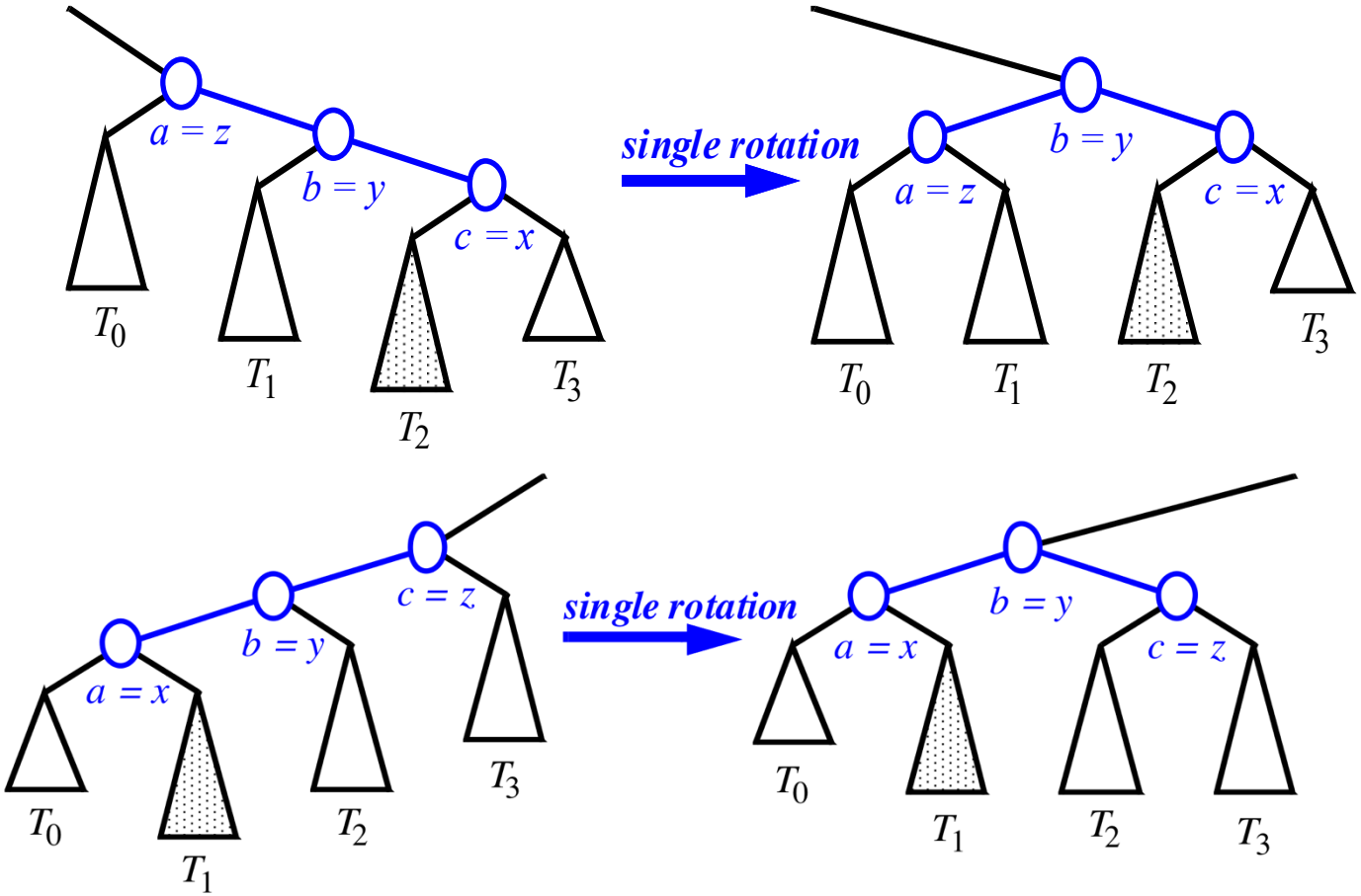
Let a, b, c be the inorder listing of x, y, z

Perform the rotations so as to make b the topmost node of the three.



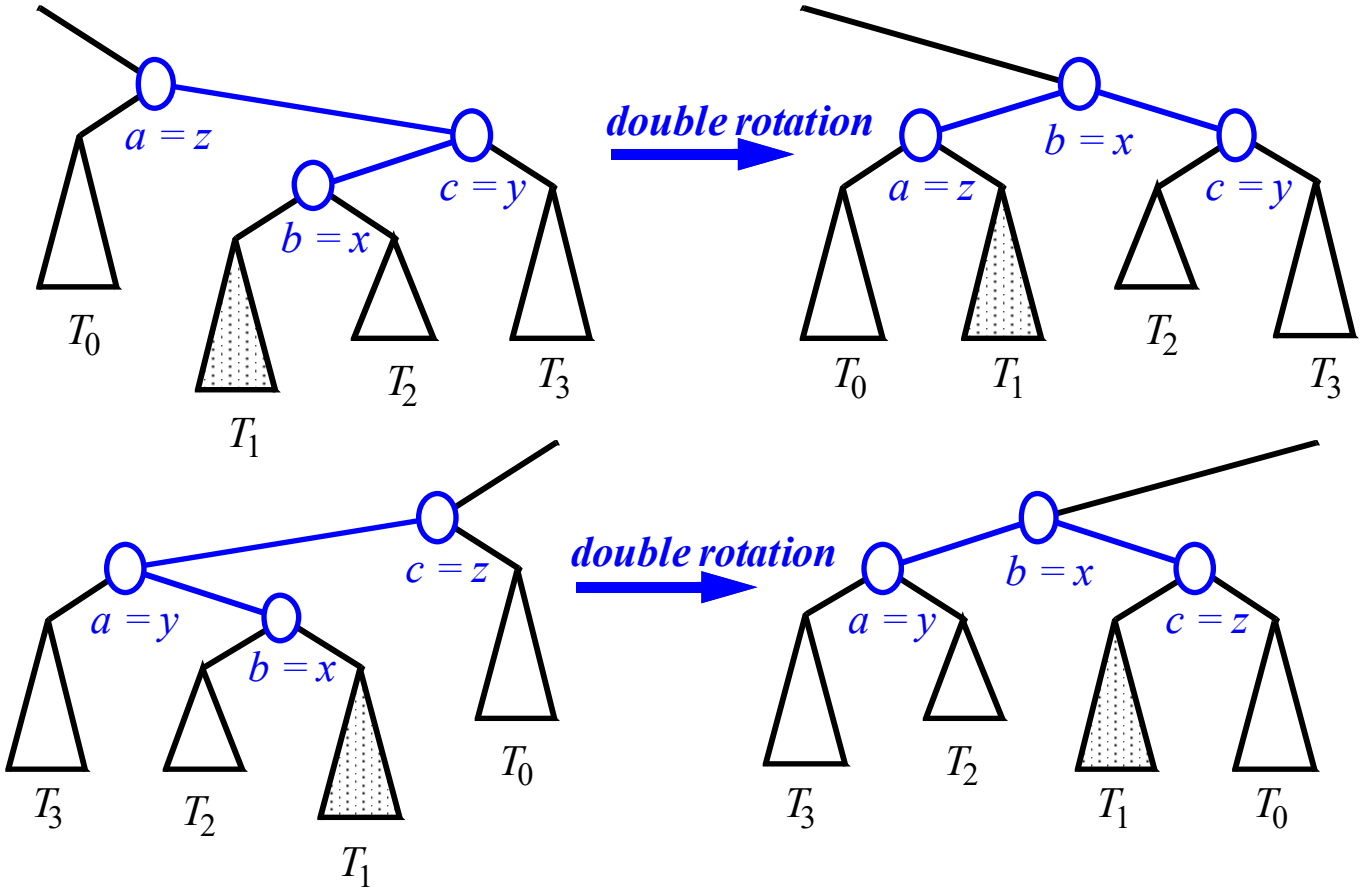
Trinode Restructuring (when done by Single Rotation)

Single Rotations:



Trinode Restructuring (when done by Double Rotation)

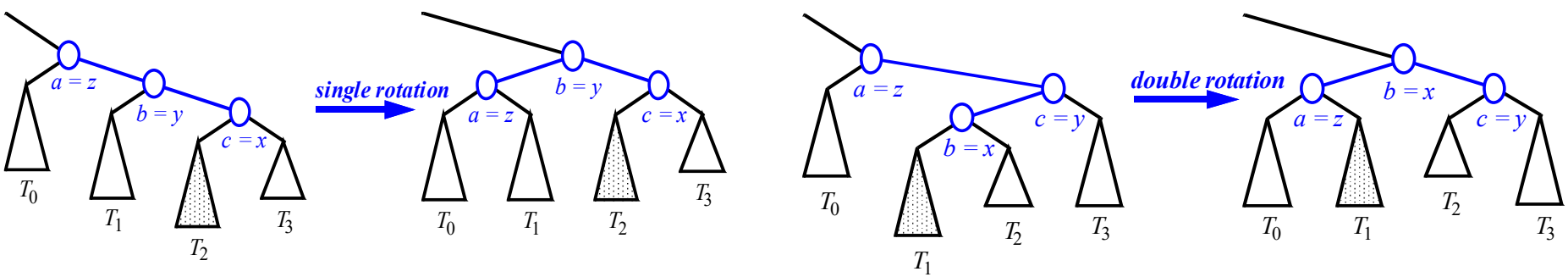
Double rotations:



Performance

Assume we are given a reference to the node x where we are performing a trinode restructure and that the binary search tree is represented using nodes and pointers to parent, left and right children

A single or double rotation takes O(1) time, because it involves updating O(1) pointers.



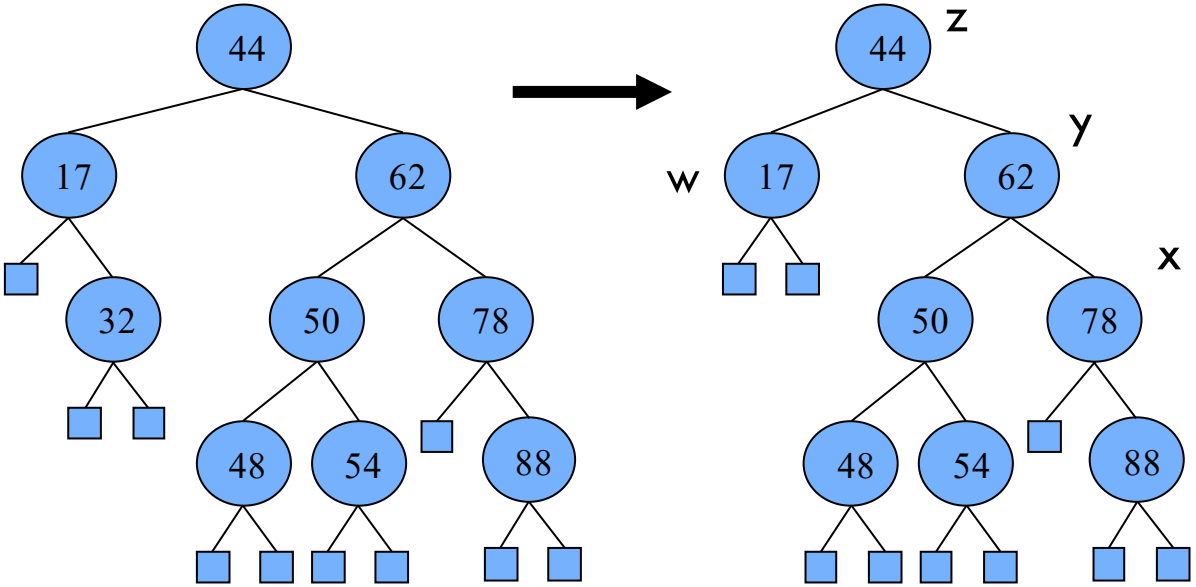
Removal in AVL trees

Suppose we are to remove a key k from our tree:

1. If k is not in the tree, search for k ends at external node There is nothing to do so tree structure does not change
2. If k is in the tree, search for k performs usual BST removal leading to removing a node with an external child and promoting its other child, which we call w
3. The new tree has BST property, but it may not have AVL balance property at some ancestor of w since
   - some ancestors of w may have decreased their height by 1
   - every node that is not an ancestor of w hasn't changed its heights
4. We use rotations to rearrange tree and re-establish AVL property, while keeping BST property

Re-establishing AVL property

- Let w be the parent of deleted node
- Let z be *lowest* ancestor of w, whose children heights differ by 2
- Let y be the child of z with larger height (y is not an ancestor of w)
- Let x be child of y with larger height

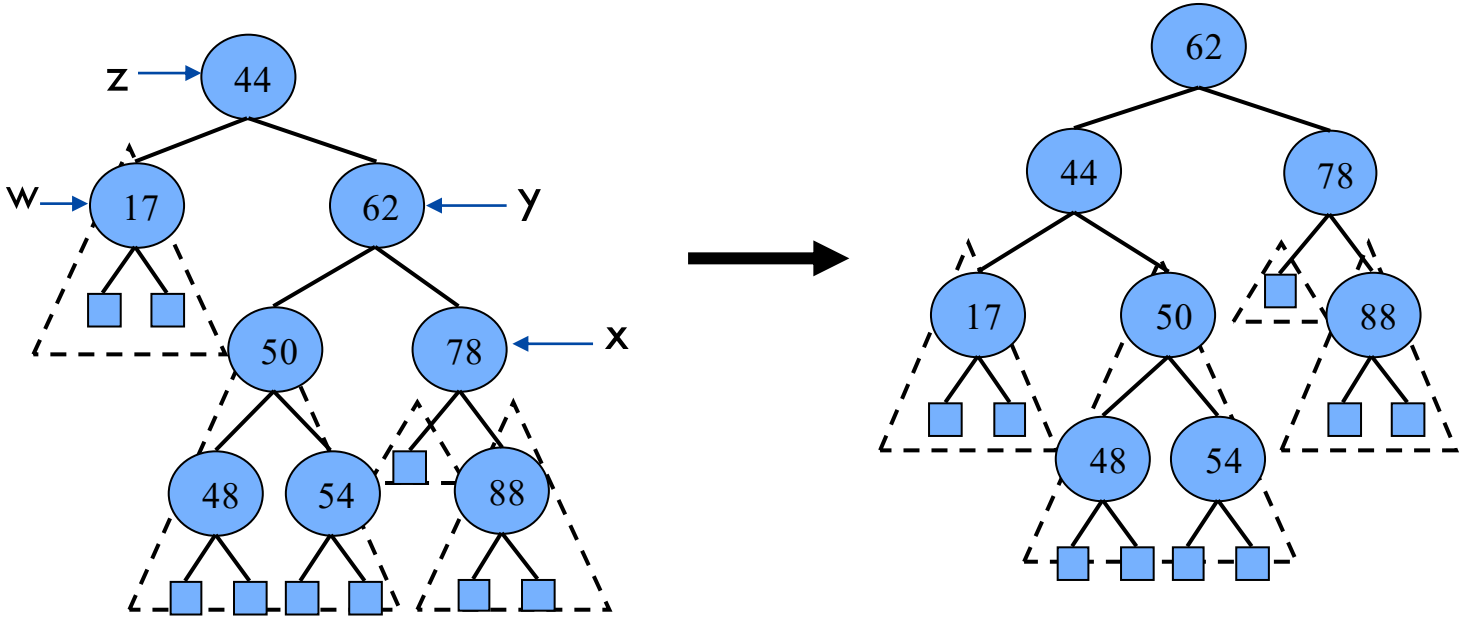


before deletion of 32

after initial deletion

Re-establishing AVL property

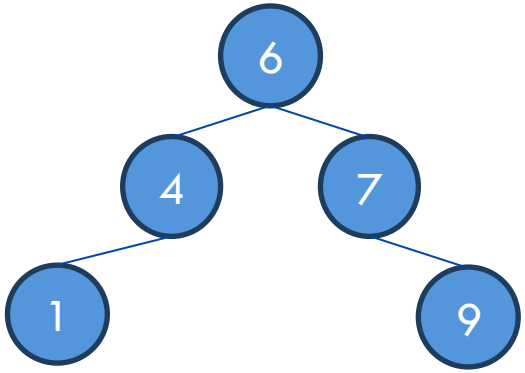
- If tree does not have AVL property, do a trinode restructure at x, y, z
- This restores the AVL property at z but it may upset the balance of another node higher in the tree, we must continue checking for balance until the root of T is reached



AVL Tree Performance

Suppose we have an AVL tree storing n items then

- The data structure uses O(n) space
- Height of the tree O(log n)
- Searching takes O(log n) time
- Insertion takes O(log n) time
- Removal takes O(log n) time



Today we just saw a sketch of how insertions and removals are performed. Working out all the details behind these operations is too heavy for the lecture, but I hope you got a flavor for what they entail and I encourage you to read the details on your own.

Discovery Activity

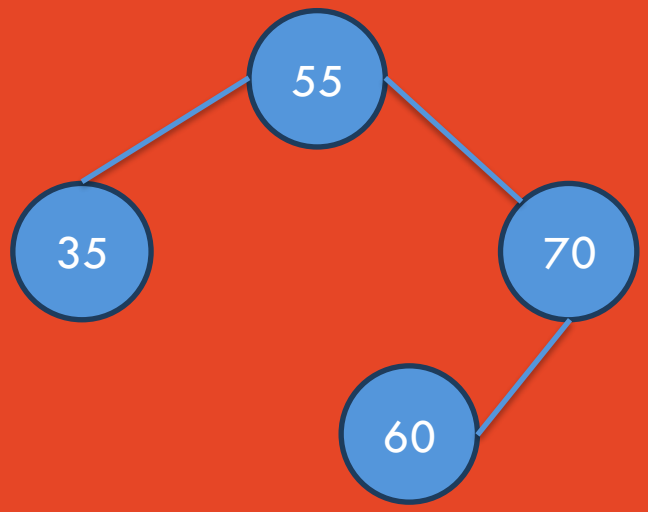
You are starting your own car dealership as a side hustle. In order to manage the inventory you use AVL trees. You start with the following cars:

- Honda Civic: $55,000
- Nissan Skyline: $70,000
- Delorean: $60,000
- Suzuki Swift: $35,000

1. Build your AVL tree using insertion
2. Your first customer purchases the Suzuki Swift update your inventory to show the resultant tree

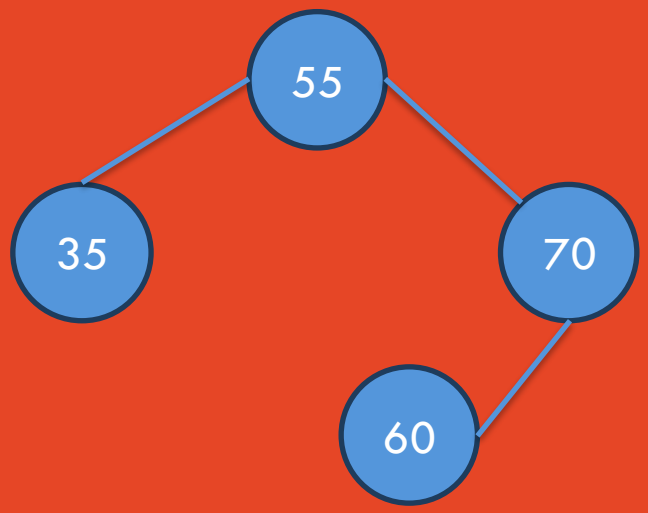
Discovery Activity

1. Insert the cars into your AVL tree (in correct order)
   - Honda Civic: $55,000
   - Nissan Skyline: $70,000
   - Delorean: $60,000
   - Suzuki Swift: $35,000



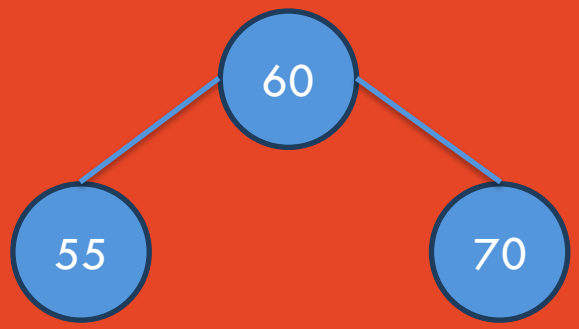
Discovery Activity

2. Your first customer purchases the Suzuki Swift update your inventory to show the resultant tree



Discovery Activity

2. Rebalance the tree



Question3 of last year's assignment1

Question 3

You are given an array $T = [t_0, t_1, \ldots, t_{n-1}]$ of n daily temperatures.

For each day x, compute how many consecutive previous days (including day x itself) had temperatures less than or equal to $T[x]$.

Return an array $S[0 \ldots n-1]$ where $S[x]$ is the span of day x.

Examples:

$$\text{Input: } T = [30, 35, 40, 38, 35, 42, 37] \quad \Rightarrow \quad S = [1, 2, 3, 1, 1, 6, 1]$$

Explanation:

- Day 0: $30 \rightarrow$ only itself $\rightarrow$ span $= 1$
- Day 1: $35 \geq 30 \rightarrow$ span $= 2$

- Day 2: $40 \geq 35 \geq 30 \rightarrow$ span $= 3$
- Day 3: $38 \geq 35 \rightarrow$ but not $40 \rightarrow$ span $= 1$
- Day 5: $42 \geq$ all before $\rightarrow$ span $= 6$

a) Design an algorithm to compute the temperature spans in $O(n)$ time. **(6 marks)**

b) Explain why your algorithm is correct. **(3 marks)**

c) Analyze the running time of your algorithm. **(3 marks)**

Solution:

a) We can solve this in $O(n)$ using a stack to keep track of the indices of days with temperatures greater than the current day's temperature.

b) The algorithm uses a monotonic decreasing stack of indices to keep track of the last temperature that is strictly greater than the current one. When popping from the stack, we remove all previous days that have temperatures less than or equal to the current temperature. The difference between the current day and the new top of the stack gives us the number of consecutive smaller/equal temperatures, i.e., the span. If the stack is empty, all previous temperatures were smaller/equal, so the span is $i + 1$.

c) Each index is pushed and popped at most once from the stack. All other operations (comparisons, arithmetic, etc.) are constant time. Hence, the algorithm runs in $\boxed{O(n)}$ time.

Fully anonymous feedback form for COMP9123 students – Fill out form